

GRAND CHALLENGE MONOGRAPH COLLECTION

TYPE THEORY

The Grand Unified Theory of Computation

COMPREHENSION

How Computational Worlds Are Built

Volume II

Grand Challenge Labs

Publication scope

This release candidate is a self-contained teaching monograph and an internally audited research artifact. It is not represented as an externally refereed final edition. The named metatheorems are scoped to the calculi stated in the chapters and in the theorem audit; executable laboratories provide regression evidence, not machine-checked proofs.

The phrase *grand unified theory* is the series research thesis, not a theorem. Volume II asks whether a small number of disciplined formation principles—family formation, context extension, dependent products, dependent sums, indexed induction, conversion, and stratified coding—can explain how computational languages enlarge the class of objects they can describe while keeping construction checkable.

Preface: From judgment to world-building

Volume I centered the judgment $\Gamma \vdash a : A$. It showed that before computation can be trusted, a language needs rules that say what expressions mean, what assumptions are available, and which reductions preserve those meanings. Volume II moves one level outward. It asks how the language obtains new types in the first place.

The decisive step is dependency:

$$\Gamma, x : A \vdash B(x) \text{ type.}$$

A type may now vary with information already present in the context. That single permission alters context structure, substitution, function spaces, records, data invariants, elimination rules, type checking, and eventually the semantics of context extension itself.

The word *comprehension* is used here in the type-theoretic and categorical sense of controlled family formation and context extension. It does not mean unrestricted set comprehension. The discipline of this volume is therefore constructive: every new expressive power arrives with formation, introduction, elimination, computation, and checking obligations.

Series orientation

I	JUDGMENT	the grammar of meaningful computation
II	COMPREHENSION	how computational worlds are built
III	PROOF / PROGRAM	logic becomes executable
IV	PROTOCOL	computation as communication
V	EFFECTS	computation meets the world
VI	UNIVERSES	languages that speak about languages
VII	IDENTITY	equality becomes structure
VIII	SEMANTICS	what makes a type theory mean something
IX	LIMITS OF CONSTRUCTION	incompleteness, partiality, boundaries
X	SYNTHESIS	the strongest thesis that survives

Contents

Publication scope	ii
Preface: From judgment to world-building	iii
Series orientation	iv
I Dependency	1
1 A Type Can Depend on a Value	2
1.1 The family judgment	2
1.2 Contexts become telescopes	3
1.3 What dependency does not mean	3
1.4 Weakening with dependency	5
1.5 Worked example: a two-index family	5
1.6 Computational Laboratory 1	6
1.7 Exercises	6
2 Substitution Changes the World	8
2.1 Substitution on terms, types, and telescopes	8
2.2 Substitution is not evaluation	9
2.3 Composition of substitutions	9
2.4 The comprehension loop	11
2.5 Computational Laboratory 2	11
2.6 Exercises	11

3	Π — One Function, Many Result Types	14
3.1	Formation	14
3.2	The ordinary arrow is the constant-family case	14
3.3	Introduction and elimination	15
3.4	Computation and binder-aware substitution	15
3.5	A first bidirectional view	17
3.6	Computational Laboratory 3	17
3.7	Exercises	17
4	Σ — Data Whose Shape Depends on Data	20
4.1	Formation and introduction	20
4.2	The ordinary product is the constant-family case	20
4.3	Projections	21
4.4	Computation	21
4.5	Existential reading without importing logic	21
4.6	Computational Laboratory 4	23
4.7	Exercises	23
5	Context Extension Is Comprehension	26
5.1	The projection that forgets the new variable	26
5.2	Exchange is conditional	26
5.3	Structural substitution as reindexing	27
5.4	Π and Σ as operations on families	27
5.5	A structural theorem schema	27
5.6	Computational Laboratory 5	29
5.7	Exercises	29
II	Indexed Construction	31
6	Indices as Static State	32
6.1	Indices rule out constructor/index mismatches	32
6.2	Static state versus runtime state	33

6.3	A small canonical-forms fact	33
6.4	From bounds to shapes	33
6.5	Computational Laboratory 6	33
6.6	Exercises	33
7	Constructors That Refine Indices	37
7.1	The constructor tells the checker the invariant	37
7.2	Selected inductive families, not arbitrary declarations	37
7.3	Canonical forms for vectors	39
7.4	Computational Laboratory 7	39
7.5	Exercises	39
8	Elimination Without Losing the Index	42
8.1	Safe lookup	42
8.2	The motive explains dependent pattern matching	44
8.3	No magic totality claim	44
8.4	Computational Laboratory 8	44
8.5	Exercises	44
9	When Equality Is Needed	47
9.1	Definitional equality is kernel policy	47
9.2	Where reduction solves the mismatch	47
9.3	Where definitional equality runs out	48
9.4	Algorithmic equality	48
9.5	Computational Laboratory 9	48
9.6	Exercises	50
III	Checking and Reflection	52
10	Checking Dependent Terms	53
10.1	A deliberately small algorithm	53
10.2	Why lambdas do not synthesize freely	55
10.3	Soundness and decidability claims	55

10.4	Computational Laboratory 10	55
10.5	Exercises	56
11	Elaboration and Missing Information	58
11.1	Metavariables as questions	58
11.2	Constraint flow	58
11.3	Elaboration may fail even when a term exists	59
11.4	Computational Laboratory 11	59
11.5	Exercises	59
12	Codes for Types, Without a Type of All Types	63
12.1	Closing codes under type formers	63
12.2	Why not Type : Type?	64
12.3	Stratification pressure	64
12.4	Computational Laboratory 12	64
12.5	Exercises	64
13	Comprehension as Language Design	68
13.1	A categorical reading: context and display map	68
13.2	Two calculi, two worlds	70
13.3	Expressiveness is not free	70
13.4	Computational Laboratory 13	70
13.5	Exercises	71
14	From Construction to Proposition	73
14.1	What the grand-unification thesis has earned	73
14.2	Indices do not guarantee the world	74
14.3	The proof/program threshold	74
14.4	A final distinction: specification versus proposition	74
14.5	Computational Laboratory 14	74
14.6	Exercises	76
	Formal reference	78

Glossary	80
Bibliography and historical notes	81
Course and self-study roadmap	83
Transition to Volume III	84

Part I

Dependency

Chapter 1

A Type Can Depend on a Value

A conventional typing judgment answers a local question: under the assumptions in Γ , is A a meaningful type, and does t inhabit it? In a simply typed calculus, later types may be complicated, but they do not change merely because an earlier term has a different value. Dependency changes exactly that constraint.

Suppose a program stores an index into an array. The useful fact is not merely “the index is a natural number.” It is “the index is smaller than the length of this array.” The second statement relates two pieces of information. If the type language cannot mention the length, the relation must live elsewhere: in a comment, a proof outside the language, a runtime check, or a programmer convention.

A dependent family allows the type itself to vary with earlier information. We begin with the schematic family $\text{Fin}(n)$. For now, Fin is opaque: when $n : \mathbb{N}$, $\text{Fin}(n)$ is a type. We postpone its constructors so that the structural consequence of dependency is visible before indexed induction enters.

Intuition

A fixed type is one room. A dependent family is a floor plan whose available room depends on where you stand. The picture is useful only if we remember that the formal object is still a judgment generated by rules; no geometric model is assumed.

1.1 The family judgment

The central move is

$$\Gamma, x : A \vdash B(x) \text{ type.}$$

The notation does not assert that B is a runtime function. It says that under a context containing $x : A$, the expression $B(x)$ is a well-formed type. Different substitutions for x may therefore specialize the family to different types.

For the first teaching kernel, COMP-0, the syntax is intentionally small:

$$A ::= \mathbb{N} \mid \text{Bool} \mid \text{Fin}(t), \quad t ::= x \mid 0 \mid \text{succ}(t) \mid \text{true} \mid \text{false}.$$

The distinctive family formation rule is

$$\frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Fin}(n) \text{ type}}.$$

Everything else in the first chapter exists to make this rule structurally legal.

1.2 Contexts become telescopes

Consider

$$n : \mathbb{N}, \quad i : \text{Fin}(n).$$

The declaration of i depends on the earlier declaration of n . The reversed sequence

$$i : \text{Fin}(n), \quad n : \mathbb{N}$$

is not merely inelegant; it is ill formed because n is unavailable when $\text{Fin}(n)$ must be checked.

This motivates the **telescope**: an ordered sequence of declarations in which each type may mention variables declared to its left.

$$\frac{}{\cdot \text{ ctx}} \quad \frac{\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type} \quad x \notin \text{dom}(\Gamma)}{\Gamma, x : A \text{ ctx}}.$$

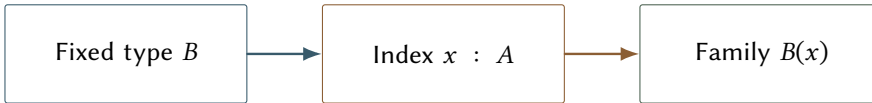
Do not confuse

A telescope is not an unordered finite map. An implementation may use a map internally, but it must preserve the dependency order needed to justify each declaration. The mathematics is about well-formed dependency, not a data-structure mandate.

1.3 What dependency does not mean

A common first intuition says that a dependent type is a type carrying a runtime tag. That can be a useful implementation technique in some languages, but it is not the judgmental meaning. Formation is static structure: it licenses a type expression under assumptions. Evaluation is an operational relation on terms. Conflating them makes later substitution rules look mysterious, because substitution changes types even when no runtime step occurs.

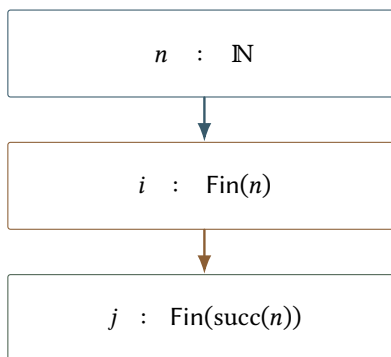
Fixed World / Dependent World



A family judgment permits type formation to depend on an earlier term.

Figure 1.1: **Plate 1. Fixed World / Dependent World.** A family judgment permits type formation to depend on an earlier term. The fiber language is geometric intuition; no set-theoretic model is assumed.

The Telescope



Context order carries dependency information.

Figure 1.2: **Plate 2. The Telescope.** Context order carries dependency information. Arrows represent free-variable dependency, not runtime dataflow.

1.4 Weakening with dependency

In a simply typed setting, weakening often feels automatic: add an unused assumption and an old derivation remains valid. Dependency forces the position of that insertion to be stated.

Theorem 1.1 (Weakening for COMP-0). *Suppose $\Gamma, \Delta \vdash J$ is derivable, $\Gamma \vdash A$ type, and x is fresh. Then*

$$\Gamma, x : A, \Delta \vdash J$$

is derivable provided the inserted declaration does not invalidate well-formedness of the trailing telescope Δ .

Proof. Proceed simultaneously by induction on the derivation of J and the formation of Δ . Base types remain well formed in any well-formed extension. Variables already present in Γ, Δ remain present. In the family case, a premise $\Gamma, \Delta \vdash n : \mathbb{N}$ is weakened by the induction hypothesis, then the Fin formation rule is reapplied. Context extension follows because each trailing declaration is rechecked in the extended prefix. No exchange principle is invoked. $\square$

The caveat is the point. Weakening preserves an existing dependency graph when a declaration is inserted at a legal place; it does not say declarations may be permuted freely.

1.5 Worked example: a two-index family

Suppose we add only the formation rule

$$\frac{\Gamma \vdash m : \mathbb{N} \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Matrix}(m, n) \text{ type}}.$$

Without defining a single matrix constructor, the language can already express contexts such as

$$m : \mathbb{N}, n : \mathbb{N}, A : \text{Matrix}(m, n).$$

The expressive gain is not that matrices suddenly compute. It is that the well-formedness of later types can depend on earlier shape information. Constructors and eliminators will determine what one can actually build and observe.

1.6 Computational Laboratory 1

Computational Laboratory

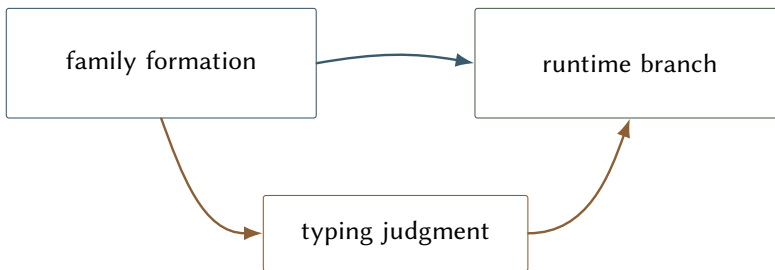
Run `labs/lab01_telescope.py`. It builds several candidate contexts, computes the free-variable dependency edges between declarations, and checks whether every edge points to an earlier declaration. The evidence records both accepted and rejected telescopes.

The lab deliberately does not infer missing declarations. If $i : \text{Fin}(n)$ appears before $n : \mathbb{N}$, the context is rejected. Elaboration comes much later; the kernel first needs a precise object to check.

1.7 Exercises

- 1.1. Checkpoint.** Decide which is well formed: $n : \mathbb{N}, i : \text{Fin}(n)$ or $i : \text{Fin}(n), n : \mathbb{N}$. Name the failed premise in the bad case.
- 1.2. Checkpoint.** Derive $n : \mathbb{N} \vdash \text{Fin}(\text{succ}(n))$ type.
- 1.3. Checkpoint.** Explain why family formation is not a runtime case split.
- 1.4. Core.** Give a four-declaration telescope containing two declarations that depend on the same natural-number variable.
- 1.5. Core.** Show by weakening that $n : \mathbb{N}, b : \text{Bool} \vdash \text{Fin}(n)$ type.
- 1.6. Core.** Construct an attempted exchange of adjacent declarations that breaks well-formedness.
- 1.7. Synthesis.** Compare a telescope with a compiler symbol table. Which implementation details may differ while the same typing invariant is preserved?
- 1.8. Synthesis.** Refine the slogan “types can contain values” into a statement that is defensible in COMP-0.
- 1.9. Proof Workshop.** Prove the variable case of weakening, distinguishing a variable declared in Γ from one declared in Δ .
- 1.10. Proof Workshop.** Prove that if $\Gamma \vdash t : \mathbb{N}$, then extending Γ by a fresh independent Boolean declaration preserves formation of $\text{Fin}(t)$.
- 1.11. Design Clinic.** Add `Matrix( $m, n$ )` only as a family former. State its formation rule and three contexts it makes expressible.
- 1.12. Challenge.** State a sufficient free-variable condition under which two adjacent declarations in a telescope may be exchanged.

Dependency Is Static Structure



Type dependency is a formation discipline, not merely dynamic tagging.

Figure 1.3: **Plate 3. Dependency Is Static Structure.** Type dependency is a formation discipline, not merely dynamic tagging. Implementations may retain runtime evidence, but the judgments remain distinct.

Chapter 2

Substitution Changes the World

In a simply typed calculus, substitution is often introduced as replacement inside a term. In a dependent calculus, that description is incomplete. A later type can mention the substituted variable, so replacing a variable changes the types under which later terms are judged.

The small context

$$n : \mathbb{N}, \quad i : \text{Fin}(n)$$

contains the entire phenomenon. Substitute 3 for n and the declaration of n disappears while the remaining declaration becomes

$$i : \text{Fin}(3).$$

The transformation is not evaluation. It is reindexing of the surrounding typing information.

2.1 Substitution on terms, types, and telescopes

Write $E[a/x]$ for capture-avoiding substitution. In COMP-0 there are no binders inside terms or types, so the definitions are structurally transparent:

$$x[a/x] = a, \quad y[a/x] = y \ (y \neq x), \quad \text{succ}(t)[a/x] = \text{succ}(t[a/x]),$$

$$\text{Fin}(t)[a/x] = \text{Fin}(t[a/x]).$$

For a trailing telescope

$$\Delta = y_1 : B_1, \dots, y_k : B_k,$$

we substitute through every declaration:

$$\Delta[a/x] = y_1 : B_1[a/x], \dots, y_k : B_k[a/x].$$

This is why the theorem requires a trailing context in its statement.

Theorem 2.1 (Generalized substitution for COMP-0). *If*

$$\Gamma, x : A, \Delta \vdash J \quad \text{and} \quad \Gamma \vdash a : A,$$

then, whenever $\Delta[a/x]$ is well formed,

$$\Gamma, \Delta[a/x] \vdash J[a/x].$$

Proof. Induct on the derivation of J , simultaneously maintaining well-formedness of the trailing telescope. The variable case has three subcases: the variable is x , lies in Γ , or lies in Δ . The first uses the premise $\Gamma \vdash a : A$; the others are preserved under reindexed context formation. Base-type cases are immediate. For $\text{Fin}(t)$ type, substitute in the premise $t : \mathbb{N}$ and reapply family formation. The context-extension case uses the induction hypothesis on the declaration type before extending the substituted prefix. $\square$

2.2 Substitution is not evaluation

If a context contains $n : \mathbb{N}$ and we reason hypothetically with a term $a : \mathbb{N}$, substituting a for n is a metatheoretic operation on syntax and judgments. It does not imply that a is evaluated first, nor that the type system is dynamically mutating at runtime. Later operational semantics may use substitution to define beta reduction, but the concepts remain distinct.

Do not confuse

Do not read $B[a/x]$ as “run a and update the type.” Read it as “specialize the family expression B by structural substitution.”

2.3 Composition of substitutions

The basic algebra matters because real derivations apply many substitutions. If $x \neq y$ and freshness conditions prevent capture, then structurally

$$E[a/x][b/y] \quad \text{and} \quad E[b/y][a[b/y]/x]$$

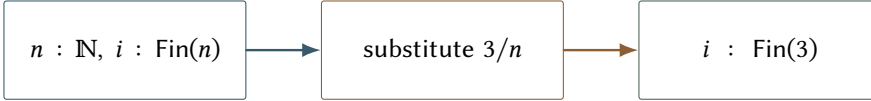
coincide. The exact equation depends on the usual side conditions; the lesson is that substitution is not an ad hoc text rewrite but an operation with compositional laws.

Proposition 2.2 (Substitution composition, binder-free fragment). *For COMP-0 expressions, if $x \neq y$ and $x \notin \text{FV}(b)$, then*

$$E[a/x][b/y] = E[b/y][a[b/y]/x].$$

Proof. Structural induction on E . The only nontrivial base cases are variables. The family case reduces to the induction hypothesis on its index term. There are no binders in COMP-0, so alpha-renaming is unnecessary here; Chapters 3–4 restore that complication. $\square$

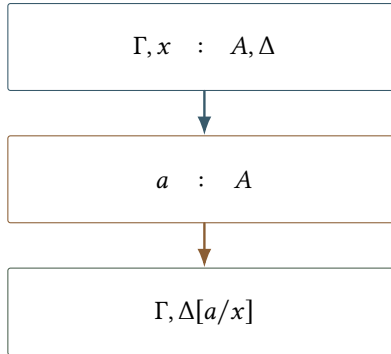
Substitution Rewrites the World



Substitution specializes later declaration types as well as terms.

Figure 2.1: **Plate 4. Substitution Rewrites the World.** Substitution specializes later declaration types as well as terms. This is structural substitution, not evaluation.

The Trailing Context



The generalized theorem must account for dependent declarations after the substituted variable.

Figure 2.2: **Plate 5. The Trailing Context.** The generalized theorem must account for dependent declarations after the substituted variable. Well-formedness side conditions remain explicit.

2.4 The comprehension loop

A family is formed under an extended context; a term is constructed under that family; substitution supplies an actual earlier value; the family and the term specialize together. This loop is the basic dynamics of dependent syntax:

form family $\rightarrow$ extend context $\rightarrow$ construct $\rightarrow$ substitute $\rightarrow$ specialize.

It is a pedagogical cycle, not a cyclic proof theory: formal derivations remain trees.

2.5 Computational Laboratory 2

Computational Laboratory

Run `labs/lab02_substitution.py`. It performs structural substitution on a telescope containing $\text{Fin}(n)$ declarations, checks that the substituted context remains well formed, and tests the binder-free composition law on a finite set of generated expressions.

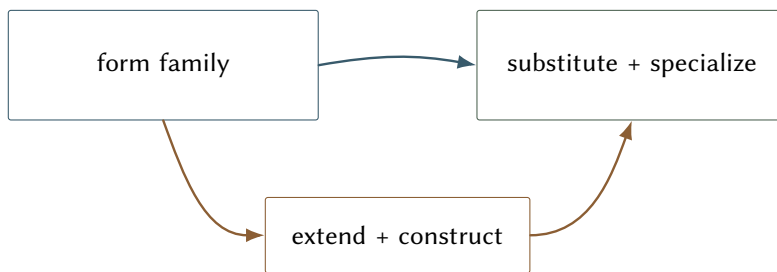
This is regression evidence for the implementation, not a mechanized proof of the theorem. The proof is the structural argument above; the executable evidence is useful because it catches implementation mistakes that prose can overlook.

2.6 Exercises

- 2.1. **Checkpoint.** Compute $(\text{Fin}(\text{succ}(n)))[0/n]$.
- 2.2. **Checkpoint.** Substitute 2 for n in $n : \mathbb{N}, i : \text{Fin}(n), j : \text{Fin}(\text{succ}(n))$.
- 2.3. **Checkpoint.** State why substitution and evaluation are distinct.
- 2.4. **Core.** Give an example where substitution changes a trailing declaration type but not the term being judged.
- 2.5. **Core.** Check the substitution-composition equation for $E = \text{Fin}(\text{succ}(x))$.
- 2.6. **Core.** Explain why a theorem stated only for $\Gamma, x : A \vdash J$ is inconvenient beneath later dependent binders.
- 2.7. **Synthesis.** Interpret generalized substitution as specialization of a family of contexts.
- 2.8. **Synthesis.** Explain which part of substitution is syntactic and which part becomes semantic only after a model is chosen.
- 2.9. **Proof Workshop.** Prove the Fin -formation case of generalized substitution.

- 2.10. Proof Workshop.** Prove substitution composition for variables and successor terms.
- 2.11. Design Clinic.** Extend the substitution function to the family $\text{Matrix}(m, n)$ and list the new test cases required.
- 2.12. Challenge.** State the capture-avoidance problem that will appear once binders are added, and give the alpha-renaming remedy.

The Comprehension Loop



Family formation and substitution form a recurring compositional cycle.

Figure 2.3: **Plate 6. The Comprehension Loop.** Family formation and substitution form a recurring compositional cycle. Formal derivations are trees; the loop is pedagogical organization.

Chapter 3

Π — One Function, Many Result Types

An ordinary function type $A \rightarrow B$ promises that every accepted input produces a result in one fixed codomain B . Dependency allows a stronger contract: the result type may vary with the actual input. The dependent product, written $\Pi(x : A).B(x)$, internalizes that family of codomains into the type of one function.

A useful example is a hypothetical function

$$f : \Pi(n : \mathbb{N}).\text{Fin}(\text{succ}(n)).$$

Applying f to 3 does not merely return something “in some finite type.” Its result type specializes to $\text{Fin}(4)$.

3.1 Formation

The type former itself is legal only when the domain is a type and the codomain family is a type under the bound variable:

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x : A \vdash B \text{ type}}{\Gamma \vdash \Pi(x : A).B \text{ type}}.$$

This is the first place where comprehension directly creates a type former: context extension exposes a family, and Π packages the whole family into a single type in the original context.

3.2 The ordinary arrow is the constant-family case

If $x \notin FV(B)$, the family does not actually vary. We write

$$A \rightarrow B \quad \text{as an abbreviation for} \quad \Pi(_ : A).B.$$

No additional theorem is needed in this presentation; the arrow is simply the constant-family notation for Π . Other presentations may take arrow and Π as separate formers and prove a correspondence instead.

3.3 Introduction and elimination

A dependent function is introduced exactly by hypothetical construction:

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x. b : \Pi(x : A). B}.$$

Application is where dependency becomes computationally visible:

$$\frac{\Gamma \vdash f : \Pi(x : A). B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B[a/x]}.$$

The codomain is specialized by the same substitution operation studied in Chapter 2.

Worked Example

Assume

$$f : \Pi(n : \mathbb{N}). \text{Fin}(\text{succ}(n)).$$

Since $2 : \mathbb{N}$, application gives

$$f2 : \text{Fin}(\text{succ}(2)).$$

The type checker need not guess the result type. It reads the family from the synthesized type of f and substitutes the checked argument into the codomain.

3.4 Computation and binder-aware substitution

The beta rule is familiar in shape:

$$(\lambda x. b) a \longrightarrow b[a/x].$$

But substitution now acts inside type formers as well as terms. For a binder y ,

$$(\Pi(y : A). B)[a/x]$$

requires the same capture-avoidance discipline as lambda abstraction. If $y = x$, substitution stops under the binder. If $y \neq x$ but $y \in FV(a)$, alpha-rewrite y before descending.

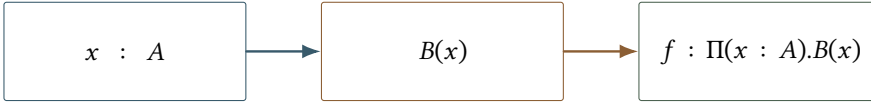
Theorem 3.1 (Principal Π -beta subject reduction). *If*

$$\Gamma \vdash (\lambda x. b) a : B[a/x],$$

derived from $\Gamma, x : A \vdash b : B$ and $\Gamma \vdash a : A$, then

$$\Gamma \vdash b[a/x] : B[a/x].$$

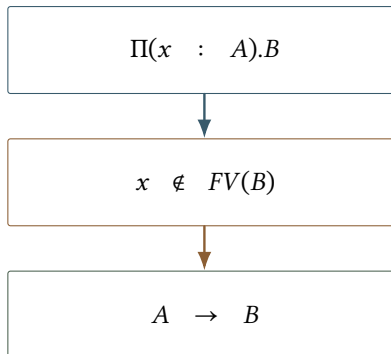
Pi as a Bundle of Codomains



A dependent function chooses an output in the codomain selected by its input.

Figure 3.1: **Plate 7. Pi as a Bundle of Codomains.** A dependent function chooses an output in the codomain selected by its input. The bundle picture is intuition, not a claim about one particular semantic model.

Constant Family: Pi Becomes Arrow



The ordinary arrow is the constant-family specialization of dependent product in this presentation.

Figure 3.2: **Plate 8. Constant Family: Pi Becomes Arrow.** The ordinary arrow is the constant-family specialization of dependent product in this presentation. Other formalisms may take arrow as primitive rather than abbreviation.

Proof. This is exactly the generalized substitution theorem lifted to the binder-aware syntax. The operational beta step removes the application-redex; substitution supplies both the reduct and its specialized result type. $\square$

The theorem proves preservation for the principal beta step. It is not, by itself, a proof of strong normalization or global type safety for every later fragment.

3.5 A first bidirectional view

Dependent application is naturally algorithmic:

$$\frac{\Gamma \vdash f \Rightarrow \Pi(x : A).B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash fa \Rightarrow B[a/x]}.$$

By contrast, an unannotated lambda generally checks against an expected Π -type rather than synthesizing one from nothing:

$$\frac{\Gamma, x : A \vdash b \Leftarrow B}{\Gamma \vdash \lambda x. b \Leftarrow \Pi(x : A).B}.$$

This division becomes the basis of Chapter 10.

3.6 Computational Laboratory 3

Computational Laboratory

Run `labs/lab03_pi.py`. The lab checks dependent application, verifies that the result type is specialized by substitution, reduces a beta redex, and tests capture avoidance under both lambda and Π binders. It intentionally refuses to synthesize a type for an unannotated lambda.

3.7 Exercises

3.1. Checkpoint. State the Π -formation rule.

3.2. Checkpoint. When may $\Pi(x : A).B$ be written as $A \rightarrow B$ in this volume?

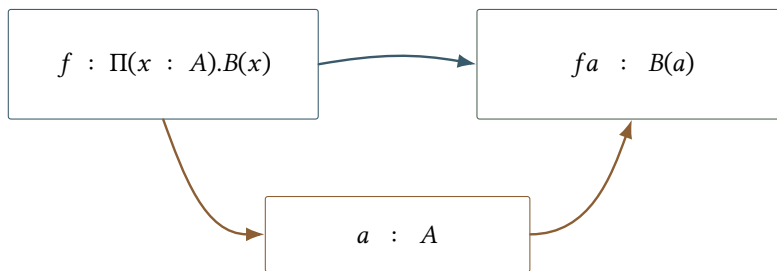
3.3. Checkpoint. If $f : \Pi(n : \mathbb{N}).\text{Fin}(\text{succ}(n))$, derive the type of $f0$.

3.4. Core. Type $\lambda n.\text{succ}(n)$ as a dependent function whose codomain happens to be constant.

3.5. Core. Give a Π -type that cannot be expressed faithfully as one fixed arrow type without hiding the dependency.

- 3.6. **Core.** Perform capture-avoiding substitution in $\lambda y.x$ with y substituted for x .
- 3.7. **Synthesis.** Explain why dependent application is simultaneously elimination and specialization.
- 3.8. **Synthesis.** Contrast “one function, many codomains” with overloading in an ordinary programming language.
- 3.9. **Proof Workshop.** Give the one-line proof of principal beta subject reduction from generalized substitution.
- 3.10. **Proof Workshop.** Show that the ordinary simply typed beta rule is recovered when the codomain is constant.
- 3.11. **Design Clinic.** Specify a dependent function type for matrix transpose that records input and output dimensions, without defining matrices themselves.
- 3.12. **Challenge.** Explain why adding function extensionality to definitional equality would materially change the checking problem, even though it sounds mathematically natural.

Application Specializes the Result



Dependent application substitutes the argument into the codomain.

Figure 3.3: **Plate 9. Application Specializes the Result.** Dependent application substitutes the argument into the codomain. Type substitution is not runtime mutation.

Chapter 4

Σ — Data Whose Shape Depends on Data

A product $A \times B$ stores two pieces of information side by side, but the type of the second component is fixed before the first is inspected. A dependent pair permits the type of the second component to depend on the first:

$$\Sigma(x : A).B(x).$$

This is the data-forming companion to Π . Where Π packages a family of codomains into one callable object, Σ packages an index together with a payload living in the corresponding fiber.

4.1 Formation and introduction

Formation mirrors Π :

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x : A \vdash B \text{ type}}{\Gamma \vdash \Sigma(x : A).B \text{ type}}.$$

A pair $\langle a, b \rangle$ inhabits this type only when the second component is checked in the specialized family:

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \langle a, b \rangle : \Sigma(x : A).B}.$$

The type of the second component is therefore not known independently of the first value.

4.2 The ordinary product is the constant-family case

When $x \notin FV(B)$, the second component does not depend on the first, and we write

$$A \times B \quad \text{for} \quad \Sigma(_ : A).B.$$

As with arrows, this is a notational specialization in the present calculus, not a claim that every formalization identifies the two by definition.

4.3 Projections

The first projection is straightforward:

$$\frac{\Gamma \vdash p : \Sigma(x : A).B}{\Gamma \vdash \text{fst } p : A}.$$

The second is dependent:

$$\frac{\Gamma \vdash p : \Sigma(x : A).B}{\Gamma \vdash \text{snd } p : B[\text{fst } p/x]}.$$

The result type remembers the first projection. This is the essential difference from an ordinary product.

4.4 Computation

For an introduced pair,

$$\text{fst } \langle a, b \rangle \longrightarrow a, \quad \text{snd } \langle a, b \rangle \longrightarrow b.$$

The second beta law is type-correct because the pair-formation premise already established $b : B[a/x]$.

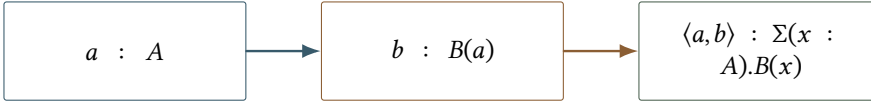
Theorem 4.1 (Principal Σ -projection preservation). *If $\Gamma \vdash \langle a, b \rangle : \Sigma(x : A).B$, then both projection beta steps preserve the types assigned by the projection rules.*

Proof. For fst , pair formation gives $\Gamma \vdash a : A$. For snd , it gives $\Gamma \vdash b : B[a/x]$. The projection typing rule assigns $B[\text{fst } \langle a, b \rangle / x]$ before reduction; conversion by the first projection beta equation identifies that with $B[a/x]$ in the current definitional equality. $\square$

4.5 Existential reading without importing logic

A dependent pair often resembles “there exists $x : A$ together with evidence in $B(x)$.” That reading becomes central in Volume III. Here we retain the computational interpretation: a Σ value is concrete data containing a first component and a second component whose type depends on it. No logical semantics is required to use the type former.

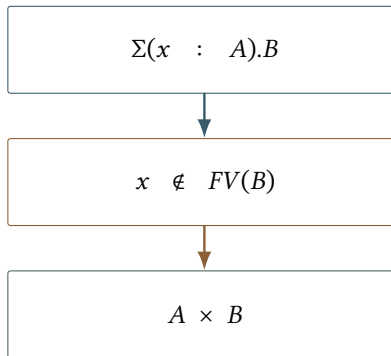
Sigma as a Fibered Pair



A dependent pair packages an index with a payload in the selected family member.

Figure 4.1: **Plate 10. Sigma as a Fibered Pair.** A dependent pair packages an index with a payload in the selected family member. The fiber picture is semantic intuition only.

Constant Family: Sigma Becomes Product



An ordinary product is the constant-family specialization of dependent sum here.

Figure 4.2: **Plate 11. Constant Family: Sigma Becomes Product.** An ordinary product is the constant-family specialization of dependent sum here. No cross-formalism claim of definitional identity is intended.

Worked Example

A length-indexed package can be written schematically as

$$\Sigma(n : \mathbb{N}).\text{Vec}(A, n).$$

A value stores a length n and a vector whose type agrees with that length. The pair is stronger than a raw natural paired with an unrelated list because the second field is checked in the fiber selected by the first.

4.6 Computational Laboratory 4

Computational Laboratory

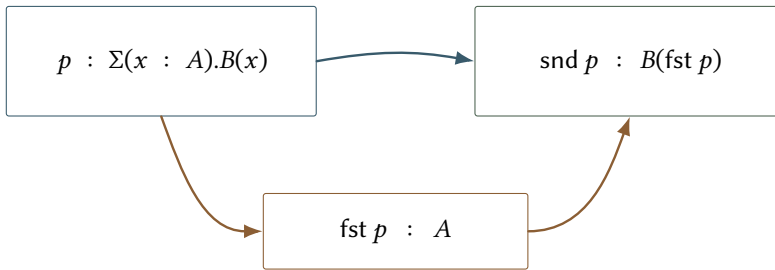
Run `labs/lab04_sigma.py`. It constructs dependent pairs, synthesizes the first projection, specializes the second-projection type, and checks both projection beta equations on a finite suite of examples.

4.7 Exercises

- 4.1. **Checkpoint.** State the Σ -formation rule.
- 4.2. **Checkpoint.** State the typing rule for the second projection.
- 4.3. **Checkpoint.** When is $\Sigma(x : A).B$ an ordinary product in this presentation?
- 4.4. **Core.** Type a pair $\langle 2, i \rangle$ assuming $i : \text{Fin}(2)$.
- 4.5. **Core.** Explain why `snd p` cannot in general be assigned one fixed type independent of `fst p`.
- 4.6. **Core.** Write the two projection beta equations.
- 4.7. **Synthesis.** Compare a dependent pair with a dynamically tagged union. What information is similar, and what is formally different?
- 4.8. **Synthesis.** Explain the computational content of the future existential reading of Σ .
- 4.9. **Proof Workshop.** Prove preservation of the first projection beta step.
- 4.10. **Proof Workshop.** Prove preservation of the second projection beta step, naming the conversion used.
- 4.11. **Design Clinic.** Design a Σ type for a matrix packaged with dimensions that its payload must respect.

4.12. Challenge. Why does a dependent record system need field-order or dependency information even when users see named fields?

The Second Projection Remembers the First



The second projection has a type specialized by the first projection.

Figure 4.3: **Plate 12. The Second Projection Remembers the First.** The second projection has a type specialized by the first projection. Dependency is typed structure, not object-oriented field lookup.

Chapter 5

Context Extension Is Comprehension

The first four chapters may look like a collection of special mechanisms: telescopes, substitution, dependent functions, dependent pairs. They are better understood as consequences of one structural operation. Given a context Γ and a type A well formed in that context, we may form the extended context

$$\Gamma.A \quad \text{or syntactically} \quad \Gamma, x : A.$$

This act of extension is **comprehension**. It turns a family over a context into a larger context in which an element of that family is available.

5.1 The projection that forgets the new variable

Every extension has an evident projection

$$\pi_A : \Gamma.A \rightarrow \Gamma$$

that forgets the new component. In raw syntax this projection is represented by weakening/substitution behavior rather than by an explicit categorical arrow. Later we will use the arrow notation as a semantic reading, not as a replacement for derivations.

The key point is that $\Gamma.A$ is not arbitrary pairing. Its second component is licensed precisely because $\Gamma \vdash A$ type. Repeating this construction builds telescopes one family at a time.

5.2 Exchange is conditional

In ordinary first-order contexts one may be tempted to reorder declarations freely. Under dependency, exchange requires independence.

Proposition 5.1 (Adjacent exchange under independence). *Suppose*

$$\Gamma, x : A, y : B, \Delta \text{ ctx}$$

and $x \notin FV(B)$, $y \notin FV(A)$, with the corresponding freshness conditions in Δ . Then the adjacent declarations may be exchanged to form

$$\Gamma, y : B, x : A, \Delta \text{ ctx},$$

after rechecking any trailing dependent types.

The condition is not cosmetic. If $B = \text{Fin}(x)$, exchange is impossible because the very formation of B requires x first.

5.3 Structural substitution as reindexing

Generalized substitution can now be read as transporting a whole telescope along a term:

$$\Gamma \vdash a : A \rightsquigarrow (\Gamma, x : A, \Delta) \mapsto (\Gamma, \Delta[a/x]).$$

The later categorical language will call this *reindexing*. At the syntactic level it is simply repeated capture-avoiding substitution together with preservation of well-formedness.

5.4 Π and Σ as operations on families

Both dependent formers consume the same input data:

$$\Gamma \vdash A \text{ type}, \quad \Gamma, x : A \vdash B \text{ type}.$$

Π returns to Γ a type of sections/functions through the family; Σ returns a type of pairs whose second component lies in the selected fiber. This is the first strong reason to treat family formation, not individual type constructors, as the organizing concept of the volume.

5.5 A structural theorem schema

The calculi in later chapters extend syntax, but the same proof obligations recur:

1. formation judgments must be stable under weakening;
2. substitution must act on every new syntactic constructor;
3. typing must be preserved by substitution;
4. computation rules must respect the specialized types produced by elimination;

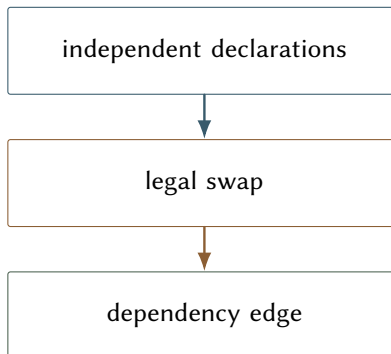
Context Extension Is Comprehension



A well-formed family extends a context
and induces a projection back to the base.

Figure 5.1: **Plate 13. Context Extension Is Comprehension.** A well-formed family extends a context and induces a projection back to the base. The arrow is a semantic reading of syntactic structure, not an extra runtime operation.

Exchange Is Conditional



Dependent contexts admit exchange only
when formation dependencies are preserved.

Figure 5.2: **Plate 14. Exchange Is Conditional.** Dependent contexts admit exchange only when formation dependencies are preserved. The diagram gives a sufficient condition, not every possible implementation optimization.

5. algorithmic checking must decide only the equality it actually implements.

This schema is more transferable than any one syntax. When a new type former is proposed, these are the first questions that determine whether it can coexist with the rest of the language.

5.6 Computational Laboratory 5

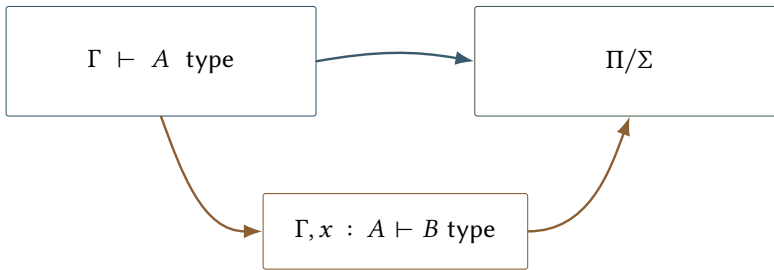
Computational Laboratory

Run `labs/lab05_context.py`. It builds a dependency graph for a telescope, computes legal adjacent exchanges, and performs reindexing by a substitution. The output distinguishes syntactically legal permutations from ones that would reverse a dependency edge.

5.7 Exercises

- 5.1. **Checkpoint.** What premise licenses the extension $\Gamma, x : A$?
- 5.2. **Checkpoint.** What does the projection $\Gamma.A \rightarrow \Gamma$ forget?
- 5.3. **Checkpoint.** State one condition that blocks exchange.
- 5.4. **Core.** Determine all legal adjacent swaps in $n : \mathbb{N}, b : \text{Bool}, i : \text{Fin}(n)$.
- 5.5. **Core.** Reindex $n : \mathbb{N}, i : \text{Fin}(n), j : \text{Fin}(\text{succ}(n))$ along $2/n$.
- 5.6. **Core.** Identify the common family data consumed by both Π and Σ formation.
- 5.7. **Synthesis.** Explain why comprehension is a better organizing idea than a list of unrelated dependent type constructors.
- 5.8. **Synthesis.** Give a programming-language analogy for context projection and state where it breaks down.
- 5.9. **Proof Workshop.** Justify adjacent exchange under the stated independence conditions.
- 5.10. **Proof Workshop.** Explain how weakening and substitution together maintain well-formed trailing contexts.
- 5.11. **Design Clinic.** Propose a new family former and list the structural obligations it creates before any evaluator is written.
- 5.12. **Challenge.** Formulate context morphisms as simultaneous substitutions and explain why composition should correspond to substitution composition.

Pi and Sigma Consume the Same Family



Both major dependent formers are operations on the same family data.

Figure 5.3: **Plate 15. Pi and Sigma Consume the Same Family.** Both major dependent formers are operations on the same family data. Their introduction, elimination, and computation rules remain different.

Part II

Indexed Construction

Chapter 6

Indices as Static State

Until now $\text{Fin}(n)$ has been an opaque family former: it illustrated dependency without saying which terms inhabit each fiber. We now let indices constrain constructors. This is the moment where a dependent family stops being only a well-formed varying type and becomes a device for recording static state.

For finite positions, use two constructors:

$$\frac{\Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{fzero} : \text{Fin}(\text{succ}(n))} \quad \frac{\Gamma \vdash i : \text{Fin}(n)}{\Gamma \vdash \text{fsucc}(i) : \text{Fin}(\text{succ}(n))}.$$

There is intentionally no constructor for $\text{Fin}(0)$.

6.1 Indices rule out constructor/index mismatches

The index is not metadata attached after construction. It is part of the constructor’s result type. fzero can inhabit $\text{Fin}(1)$, $\text{Fin}(2)$, and so on through different choices of n , but it cannot inhabit $\text{Fin}(0)$. Likewise, fsucc lifts an existing finite index into a strictly larger bound.

This is a general pattern: constructors *refine* indices. A family is useful when its constructor result types expose facts the eliminator can later rely on.

Do not confuse

“Invalid states are unrepresentable” is always relative to the invariants encoded by the family. $\text{Fin}(n)$ can encode an upper-bound discipline. It does not by itself say anything about memory safety, concurrency, numerical stability, authorization, or the semantics of the array being indexed.

6.2 Static state versus runtime state

An index such as n is static in the typing judgment, but a program may still carry a runtime representation related to it. The distinction is not “compile time versus runtime” in every implementation. It is logical role: the index appears in the type and constrains which terms are admitted. Erasure, retention, and runtime checking are implementation choices.

6.3 A small canonical-forms fact

Lemma 6.1 (Canonical forms for finite indices). *In the selected COMP-I fragment, a closed value $v : \text{Fin}(\text{succ}(n))$ is either fzero or $\text{fsucc}(v')$ for some value $v' : \text{Fin}(n)$.*

Proof. By inspection of the value constructors admitted for Fin . No other value form has a Fin result type. This is a constructor inversion lemma, not a general normalization theorem. $\square$

The qualification matters: canonical-forms lemmas classify values already known to have a type. To conclude that every closed well-typed term eventually becomes such a value would require a progress/normalization argument for the operational calculus.

6.4 From bounds to shapes

A single natural index can describe more than a bound. It can record length, protocol phase, stack depth, dimensionality, or any other static state for which the language provides appropriate constructors and eliminators. The index does not need to be a natural number in general; later systems use finite labels, trees, permissions, effects, or richer terms.

6.5 Computational Laboratory 6

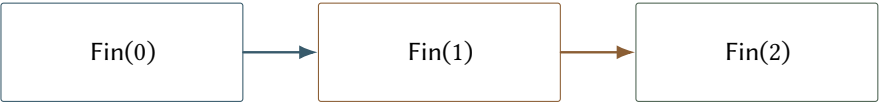
Computational Laboratory

Run `labs/lab06_fin.py`. It generates all canonical inhabitants of $\text{Fin}(n)$ for small n , checks that exactly n inhabitants are produced, and verifies constructor refinement of the bound. The finite enumeration is evidence for the teaching implementation, not a semantic definition of all possible presentations of Fin .

6.6 Exercises

6.1. Checkpoint. Why is there no constructor for $\text{Fin}(0)$?

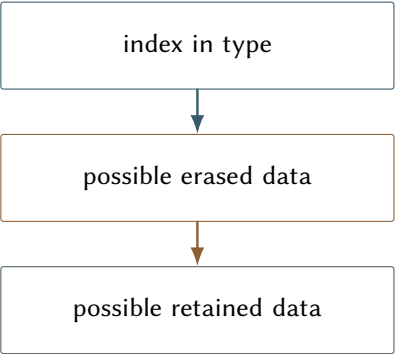
Fin as an Indexed Boundary



Constructor result indices make the finite bound part of admissible construction.

Figure 6.1: **Plate 16. Fin as an Indexed Boundary.** Constructor result indices make the finite bound part of admissible construction. The family encodes one bound invariant, not general program safety.

Static Index / Runtime Representation

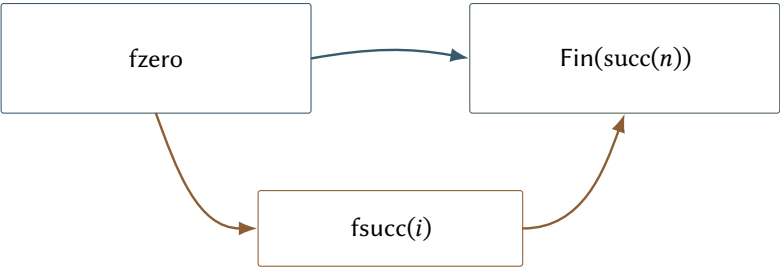


Static role and runtime representation are distinct design questions.

Figure 6.2: **Plate 17. Static Index / Runtime Representation.** Static role and runtime representation are distinct design questions. The plate does not prescribe erasure or retention.

- 6.2. **Checkpoint.** Give the type of $\text{fsucc}(\text{fzero})$ at its smallest possible bound.
- 6.3. **Checkpoint.** What fact is recorded by the index of $\text{Fin}(n)$?
- 6.4. **Core.** Enumerate the canonical inhabitants of $\text{Fin}(3)$ using the two constructors.
- 6.5. **Core.** Show that a closed value of $\text{Fin}(1)$ must be fzero .
- 6.6. **Core.** Give an invariant that $\text{Fin}(n)$ does not encode.
- 6.7. **Synthesis.** Explain why constructor result types, not prose documentation, make the bound discipline part of the language.
- 6.8. **Synthesis.** Compare an indexed state with a runtime assertion. When might an implementation still retain a runtime check?
- 6.9. **Proof Workshop.** Prove the canonical-forms lemma for $\text{Fin}(\text{succ}(n))$ by inversion on values.
- 6.10. **Proof Workshop.** Prove there is no closed canonical value of $\text{Fin}(0)$ in the selected constructor grammar.
- 6.11. **Design Clinic.** Design a three-phase protocol family indexed by a finite phase label. State only constructor result types.
- 6.12. **Challenge.** Explain why canonical forms plus preservation is still insufficient to prove termination.

Canonical Fin Values



Finite-index values expose constructor shape constrained by the result index.

Figure 6.3: **Plate 18. Canonical Fin Values.** Finite-index values expose constructor shape constrained by the result index. Canonical forms classify values; they do not prove termination.

Chapter 7

Constructors That Refine Indices

The archetypal indexed datatype is a vector whose length appears in its type:

$$\text{Vec}(A, n).$$

Lists and vectors may share a runtime representation, but their constructors state different static contracts. A list constructor says “cons adds an element.” A vector constructor additionally says exactly how the length index changes.

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash \text{vnil} : \text{Vec}(A, 0)}$$
$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash xs : \text{Vec}(A, n)}{\Gamma \vdash \text{vcons}(a, xs) : \text{Vec}(A, \text{succ}(n))}.$$

7.1 The constructor tells the checker the invariant

The index is not inferred from an external theorem about list length. It is produced by the type of the constructor itself. Therefore a term such as

$$\text{vcons}(a, \text{vnil})$$

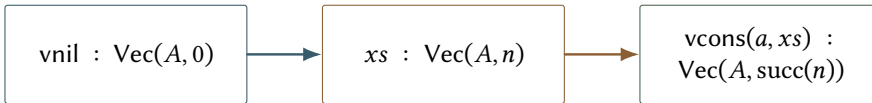
can only receive a vector type whose index is definitionally 1 in this fragment.

A mismatch such as trying to assign `vnil` the type $\text{Vec}(A, \text{succ}(n))$ fails by constructor inversion unless the equality theory identifies 0 with $\text{succ}(n)$, which this calculus does not.

7.2 Selected inductive families, not arbitrary declarations

A production dependent type theory must specify positivity, recursive occurrence, elimination principles, and termination conditions for user-defined inductive families. Volume II

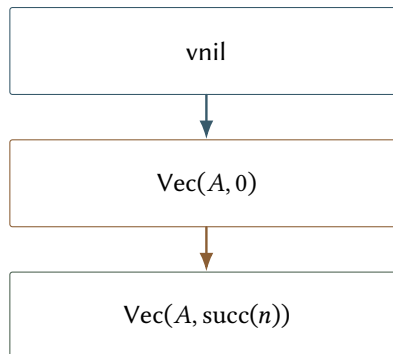
Vector Length in the Type



Vector constructors refine the length index by construction.

Figure 7.1: **Plate 19. Vector Length in the Type.** Vector constructors refine the length index by construction. Runtime layout may still coincide with ordinary lists.

Impossible Constructor / Index Combinations



Result indices rule out constructor shapes inconsistent with the claimed length.

Figure 7.2: **Plate 20. Impossible Constructor / Index Combinations.** Result indices rule out constructor shapes inconsistent with the claimed length. The conclusion depends on the current definitional equality.

does not pretend to solve the general declaration problem. It admits `Fin` and `Vec` as selected strictly positive families with explicit rules, enough to study the interaction among indices, constructors, elimination, and checking.

This scope boundary prevents a common pedagogical error: writing one intuitive datatype declaration and silently assuming all metatheory of a mature proof assistant.

7.3 Canonical forms for vectors

Lemma 7.1 (Vector shape from the index). *A closed canonical value of type $\text{Vec}(A, 0)$ is `vnil`. A closed canonical value of type $\text{Vec}(A, \text{succ}(n))$ has the form `vcons(a, xs)` with $a : A$ and $xs : \text{Vec}(A, n)$.*

Proof. Inspect the result indices of the two admitted vector constructors. The constructors are disjoint by their result-index shapes under the current definitional equality. $\square$

The lemma is precisely what makes later total operations such as vector head possible: the index excludes the `nil` constructor before evaluation begins.

7.4 Computational Laboratory 7

Computational Laboratory

Run `labs/lab07_vec.py`. It builds vectors from the two constructors, computes their index from construction history, rejects deliberately mismatched length annotations, and records the constructor form available at index 0 versus `succ(n)`.

7.5 Exercises

7.1. Checkpoint. Give the type of `vnil`.

7.2. Checkpoint. Give the type of `vcons(a, xs)` from the premises $a : A$ and $xs : \text{Vec}(A, n)$.

7.3. Checkpoint. Why can a value of $\text{Vec}(A, 0)$ not be a `vcons` under the current equality theory?

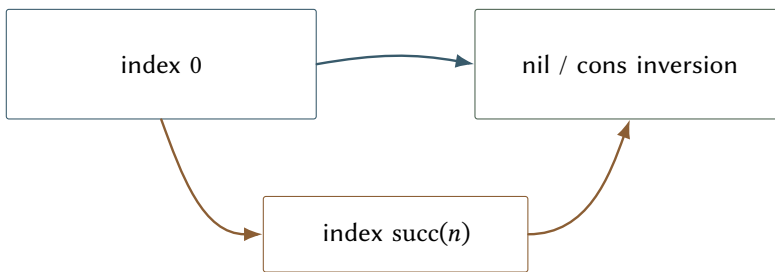
7.4. Core. Type a three-element vector explicitly from the constructors.

7.5. Core. Show the failed inversion when `vnil` is claimed to inhabit $\text{Vec}(A, 2)$.

7.6. Core. Compare the constructor types of a list and a vector.

- 7.7. Synthesis.** Explain how constructor result indices encode an invariant without a separate length theorem.
- 7.8. Synthesis.** Why is the selected-family discipline scientifically cleaner than claiming a general positivity checker?
- 7.9. Proof Workshop.** Prove the zero-length branch of the vector canonical-forms lemma.
- 7.10. Proof Workshop.** Prove the successor-length branch by constructor inversion.
- 7.11. Design Clinic.** Design an indexed binary tree family whose index records height exactly; state constructor result types and note one design difficulty.
- 7.12. Challenge.** Explain how definitional equality of indices can affect constructor inversion.

Shape from the Index



Canonical forms let an eliminator infer
which constructor shapes remain possible.

Figure 7.3: **Plate 21. Shape from the Index.** Canonical forms let an eliminator infer which constructor shapes remain possible. This is inversion on values, not global evaluation behavior.

Chapter 8

Elimination Without Losing the Index

Indexed constructors are useful only if eliminators preserve the information they establish. Ordinary pattern matching often returns one fixed result type. Dependent elimination allows the result type itself to vary with the constructor index. The family describing that varying result is traditionally called the **motive**.

For many programming examples we can see the benefit before presenting a completely general eliminator. Consider vector head:

$$\text{head} : \Pi(n : \mathbb{N}). \text{Vec}(A, \text{succ}(n)) \rightarrow A.$$

There is no nil case because the input type rules it out.

8.1 Safe lookup

An even stronger example is indexing:

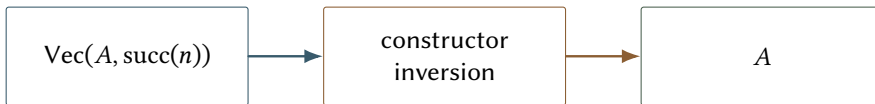
$$\text{lookup} : \Pi(n : \mathbb{N}). \text{Vec}(A, n) \rightarrow \text{Fin}(n) \rightarrow A.$$

The vector length and finite bound are the same index. The impossible cases align automatically: when the vector is empty, the index argument would need type $\text{Fin}(0)$, which has no canonical inhabitant.

A structurally recursive definition follows the indices:

$$\begin{aligned} \text{lookup } (\text{vcons}(a, xs)) \text{ fzero} &\longrightarrow a, \\ \text{lookup } (\text{vcons}(a, xs)) \text{ (fsucc}(i)) &\longrightarrow \text{lookup } xs \ i. \end{aligned}$$

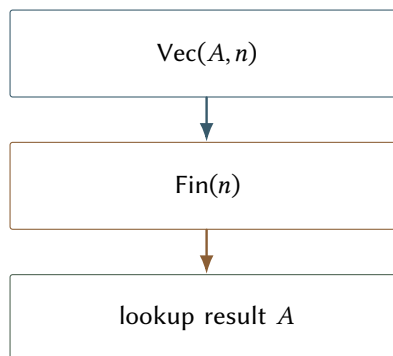
Vector Head Needs No Empty Branch



A nonempty vector type excludes the nil constructor before the head body is checked.

Figure 8.1: **Plate 22. Vector Head Needs No Empty Branch.** A nonempty vector type excludes the nil constructor before the head body is checked. The guarantee is local to the represented invariant.

Safe Lookup Aligns Two Indices



Shared indices align vector shape with admissible positions.

Figure 8.2: **Plate 23. Safe Lookup Aligns Two Indices.** Shared indices align vector shape with admissible positions. The index does not certify external memory or foreign data.

8.2 The motive explains dependent pattern matching

For a general indexed family $F(i)$, an eliminator cannot merely say “return C .” Different indices may require different result types. A motive is a family

$$i : I, x : F(i) \vdash P(i, x) \text{ type}$$

that states what must be produced for every constructor case. Each branch is checked against the motive specialized to that constructor’s result index.

This is why dependent pattern matching is more than syntax sugar for ordinary case analysis. The checker must transport index information from the constructor into the expected branch type.

8.3 No magic totality claim

A function can be free of an explicit failure case because its input type excludes that case. This is a local theorem about the represented invariant. It does not prove the function is efficient, secure, terminating under added general recursion, or semantically correct for an external specification.

Proposition 8.1 (No empty-vector head case). *In COMP-I, a closed canonical argument to head has type $\text{Vec}(A, \text{succ}(n))$ and therefore has constructor form $\text{vcons}(a, xs)$. Hence a defining equation for head requires no vnil branch.*

This proposition depends only on vector canonical forms. It is deliberately weaker than a global theorem that every closed program calling head evaluates successfully.

8.4 Computational Laboratory 8

Computational Laboratory

Run `labs/lab08_lookup.py`. The lab enumerates small vectors and finite indices, evaluates lookup, checks that every generated index is in range, and demonstrates that the empty-vector lookup input space has zero index inhabitants.

8.5 Exercises

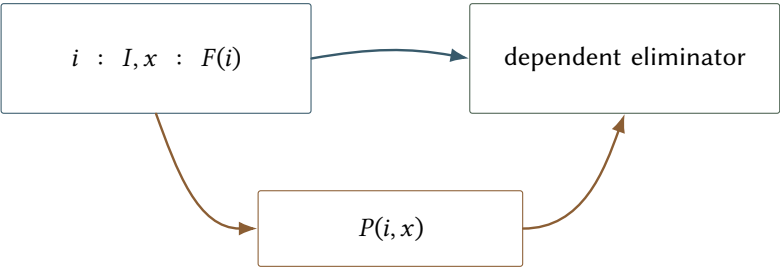
8.1. Checkpoint. State a type for vector head that excludes the empty case.

8.2. Checkpoint. State the type of safe lookup.

8.3. Checkpoint. What is a motive?

- 8.4. **Core.** Reduce lookup on a two-element vector at `fzero`.
- 8.5. **Core.** Reduce lookup on the same vector at `fsucc(fzero)`.
- 8.6. **Core.** Explain why lookup on an empty vector has no canonical well-typed index argument.
- 8.7. **Synthesis.** Compare dependent elimination with ordinary case analysis.
- 8.8. **Synthesis.** Explain exactly what totality guarantee the vector-head type provides and what it does not.
- 8.9. **Proof Workshop.** Derive the no-empty-vector-head proposition from vector canonical forms.
- 8.10. **Proof Workshop.** Show that the recursive lookup branch decreases both vector length and finite-index structure in lockstep.
- 8.11. **Design Clinic.** Specify a safe matrix row-selection function using a finite row index.
- 8.12. **Challenge.** Give an example where an indexed family requires a genuinely dependent motive rather than one fixed result type.

A Motive Preserves Dependency



A motive states the result family of dependent elimination.

Figure 8.3: **Plate 24. A Motive Preserves Dependency.** A motive states the result family of dependent elimination. The plate suppresses the full constructor-specific eliminator rules.

Chapter 9

When Equality Is Needed

Dependency turns equality from a peripheral concern into infrastructure. If a type contains a term, then deciding whether two types match may require deciding whether their indices are definitionally equal. A vector of length $\text{succ}(0)$ should normally be usable where a vector of length 1 is expected if 1 is merely notation for $\text{succ}(0)$. More interestingly, computations in indices may expose equal normal forms.

The conversion rule states the principle:

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A \equiv B \text{ type}}{\Gamma \vdash t : B}.$$

9.1 Definitional equality is kernel policy

In this volume, $\equiv$ is the compatible equivalence generated by alpha-equivalence and the computation rules explicitly admitted in the current fragment. It is not arbitrary mathematical equality. It does not include function extensionality, proof irrelevance, or user-proved propositions unless a later calculus explicitly internalizes and connects those notions.

This makes definitional equality a language-design decision. A richer equality can make more programs type-check without explicit coercions, but it can also make checking harder or undecidable. The design objective is not “maximize equality.” It is “choose an equality whose computational consequences are understood.”

9.2 Where reduction solves the mismatch

For a beta-redex in an index, normalization may reveal the same family member. Suppose an index expression contains

$$(\lambda x. \text{succ}(x)) 0.$$

Beta reduction yields $\text{succ}(0)$, so a family such as $\text{Vec}(A, (\lambda x. \text{succ}(x))0)$ is definitionally equal to $\text{Vec}(A, \text{succ}(0))$ under compatible congruence.

9.3 Where definitional equality runs out

Consider an addition operation defined by recursion on its first argument. Depending on the chosen equations, $0 + n$ may reduce definitionally to n while $n + 0$ may be stuck when n is a variable. Mathematically they are equal, but the kernel may not identify them by computation alone.

If a program has

$$xs : \text{Vec}(A, n + 0)$$

and needs $\text{Vec}(A, n)$, one reaches a *transport pressure point*. Either the equality theory is strengthened, the program carries explicit evidence and transports along it, or the representation is redesigned. Volume II stops before introducing identity types; that machinery belongs to Volume VII.

Do not confuse

Do not silently convert propositional equality into definitional equality. The former is evidence inside a type theory; the latter is part of the judgmental checking relation. Conflating them hides both computational cost and proof obligations.

9.4 Algorithmic equality

A checker needs an executable approximation/decision procedure. COMP-CHECK deliberately adopts a narrow algorithm: reduce the beta/projection redexes in its explicit core and compare the resulting syntax up to alpha-equivalence. It does not attempt arbitrary theorem proving while checking types.

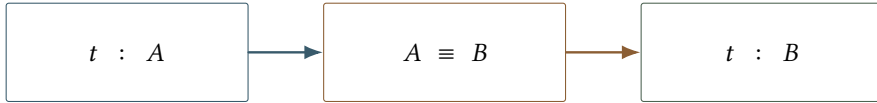
The resulting algorithm can be sound for the definitional equality implemented without being complete for every mathematically valid equality one might wish to use.

9.5 Computational Laboratory 9

Computational Laboratory

Run `labs/lab09_equality.py`. It normalizes the small explicit index language, compares examples where reduction exposes equality, and records pressure cases that remain unequal because no corresponding computation rule is admitted.

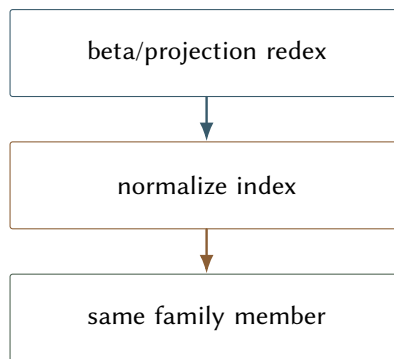
Conversion Through Definitional Equality



Conversion permits typing across equal types according to the kernel equality.

Figure 9.1: **Plate 25. Conversion Through Definitional Equality.** Conversion permits typing across equal types according to the kernel equality. Definitional equality is not arbitrary mathematical equality.

Computation Can Expose Equal Indices



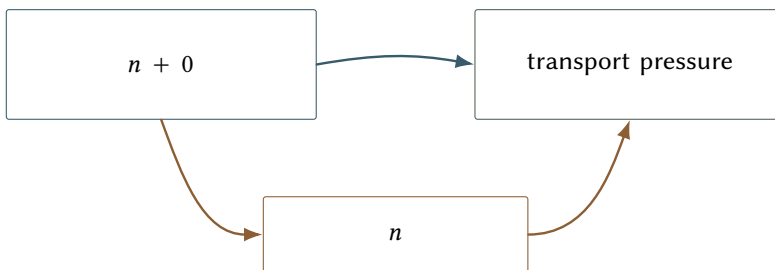
Index computation can discharge type mismatches automatically.

Figure 9.2: **Plate 26. Computation Can Expose Equal Indices.** Index computation can discharge type mismatches automatically. Only the admitted computation rules contribute.

9.6 Exercises

- 9.1. **Checkpoint.** State the conversion rule.
- 9.2. **Checkpoint.** Name two principles not included in the volume's definitional equality.
- 9.3. **Checkpoint.** Why can index computation affect type checking?
- 9.4. **Core.** Normalize $(\lambda x.\text{succ}(x))0$ and explain the induced vector-type conversion.
- 9.5. **Core.** Give a mathematical equality that need not be definitional in a chosen recursive presentation.
- 9.6. **Core.** Explain the difference between alpha-equivalence and propositional equality.
- 9.7. **Synthesis.** Why is definitional equality a language-design budget rather than merely a mathematical relation?
- 9.8. **Synthesis.** Describe three possible responses when a useful mathematical equality is not definitional.
- 9.9. **Proof Workshop.** Show that the principal Π beta equality is stable under formation of $\text{Vec}(A, -)$ by congruence.
- 9.10. **Proof Workshop.** Explain why conversion soundness depends on the equality relation preserving type formation.
- 9.11. **Design Clinic.** Choose an equality policy for a small tensor-shape language and justify what you normalize automatically.
- 9.12. **Challenge.** Analyze the trade-off of adding eta equality for functions to the kernel's algorithmic equality.

Where Definitional Equality Runs Out



Mathematical equality may exceed automatic definitional equality.

Figure 9.3: **Plate 27. Where Definitional Equality Runs Out.** Mathematical equality may exceed automatic definitional equality. Identity types and explicit transport are deferred to Volume VII.

Part III

Checking and Reflection

Chapter 10

Checking Dependent Terms

Dependent typing raises a practical question that simple rule presentations can postpone: which direction does information flow during checking? If the checker sees a variable, an annotation, or an application, it can often synthesize a type. If it sees a lambda or a dependent pair, an expected type may be needed to know how to check the body or second component. Bidirectional typing makes this asymmetry explicit.

We write

$$\Gamma \vdash t \Rightarrow A \quad \text{and} \quad \Gamma \vdash t \Leftarrow A$$

for synthesis and checking.

10.1 A deliberately small algorithm

COMP-CHECK is not intended to reproduce Agda, Coq, Lean, Idris, or any other production implementation. It is an executable teaching fragment with explicit annotations, Π , Σ , base types, and selected indexed families. The design objective is that every algorithmic rule has an obvious recursive structure.

Representative synthesis rules are

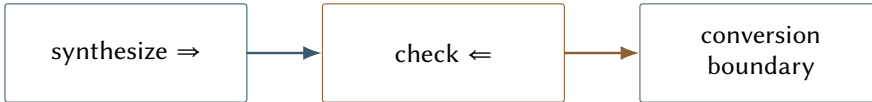
$$\frac{x : A \in \Gamma}{\Gamma \vdash x \Rightarrow A} \quad \frac{\Gamma \vdash t \Leftarrow A}{\Gamma \vdash (t : A) \Rightarrow A}$$
$$\frac{\Gamma \vdash f \Rightarrow \Pi(x : A).B \quad \Gamma \vdash a \Leftarrow A}{\Gamma \vdash fa \Rightarrow B[a/x]}.$$

Checking uses an expected type:

$$\frac{\Gamma, x : A \vdash b \Leftarrow B}{\Gamma \vdash \lambda x. b \Leftarrow \Pi(x : A).B}$$
$$\frac{\Gamma \vdash a \Leftarrow A \quad \Gamma \vdash b \Leftarrow B[a/x]}{\Gamma \vdash \langle a, b \rangle \Leftarrow \Sigma(x : A).B}.$$

A final conversion rule allows a synthesized type to be checked against a definitionally equal expected type.

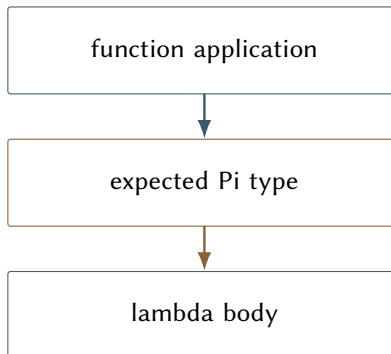
Bidirectional Information Flow



Bidirectional typing makes the direction of available information explicit.

Figure 10.1: **Plate 28. Bidirectional Information Flow.** Bidirectional typing makes the direction of available information explicit. It is an algorithmic presentation, not a new declarative truth relation.

Application Synthesizes / Lambda Checks



Applications can synthesize specialized codomains while lambdas use expected domains.

Figure 10.2: **Plate 29. Application Synthesizes / Lambda Checks.** Applications can synthesize specialized codomains while lambdas use expected domains. More complete inference procedures are possible but not assumed.

10.2 Why lambdas do not synthesize freely

From the term $\lambda x.x$ alone, many dependent types are possible. It could be an identity function on naturals, booleans, vectors, or a family of types described elsewhere. Requiring an expected Π -type avoids global inference. The checker learns the domain from the expectation, extends the context, and checks the body against the specialized codomain.

This is not a weakness of dependent typing; it is an explicit information contract between syntax and algorithm. Annotations can move a term from checking mode to synthesis mode where needed.

10.3 Soundness and decidability claims

The executable checker makes a narrow claim.

Theorem 10.1 (Algorithmic soundness, COMP-CHECK). *For every rule implemented by the COMP-CHECK teaching checker, if $\Gamma \vdash t \Rightarrow A$ succeeds then the corresponding declarative judgment $\Gamma \vdash t : A$ is derivable in the admitted core; if $\Gamma \vdash t \Leftarrow A$ succeeds, the same declarative judgment is derivable up to the implemented definitional equality.*

Proof. By induction on the successful algorithmic derivation. Each synthesis/checking rule is a syntax-directed presentation of a declarative rule or of declarative conversion. The application case uses substitution in the codomain; the lambda and pair cases use the corresponding introduction rules. $\square$

The checker terminates because the syntax-directed recursion descends through finite input syntax and delegates equality only to the bounded explicit normalizer defined for the fragment. We do not claim completeness for arbitrary declarative terms or arbitrary mathematical equality.

10.4 Computational Laboratory 10

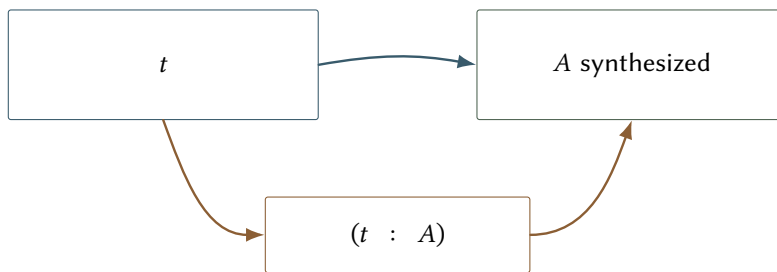
Computational Laboratory

Run `labs/lab10_checker.py`. It checks/synthesizes a curated suite of Π , Σ , `Fin`, and `Vec` examples, verifies dependent result specialization, and records expected failures for unannotated synthesis and index mismatches.

10.5 Exercises

- 10.1. Checkpoint.** Which judgment synthesizes a type and which checks against an expected type?
- 10.2. Checkpoint.** Why is application naturally a synthesis rule?
- 10.3. Checkpoint.** Why does an unannotated lambda usually check rather than synthesize?
- 10.4. Core.** Derive the algorithmic type of fa from a synthesized dependent function type.
- 10.5. Core.** Show how an annotation lets a lambda participate in a synthesis context.
- 10.6. Core.** Give a dependent pair that must be checked against an expected Σ type.
- 10.7. Synthesis.** Explain bidirectionality as information routing rather than a new notion of truth.
- 10.8. Synthesis.** Identify one declaratively typable term that the deliberately incomplete checker might reject without extra annotation.
- 10.9. Proof Workshop.** Prove algorithmic soundness for the application rule.
- 10.10. Proof Workshop.** Prove algorithmic soundness for lambda checking.
- 10.11. Design Clinic.** Decide which constructs in a small dependent record language should synthesize and which should check.
- 10.12. Challenge.** Explain why completeness of elaboration is a materially stronger claim than soundness of the core checker.

Annotations Move Information



An explicit annotation converts a checking obligation into a locally synthesizable type.

Figure 10.3: **Plate 30. Annotations Move Information.** An explicit annotation converts a checking obligation into a locally synthesizable type. Annotations are evidence for the checker, not proof of correctness by themselves.

Chapter 11

Elaboration and Missing Information

Core calculi are explicit because explicitness makes metatheory legible. Human-facing languages are rarely so austere. Programmers omit type arguments, binder types, universe levels, and coercions. An **elaborator** attempts to reconstruct that missing information and produces an explicit term for a smaller trusted checker.

The architectural separation matters:

surface syntax $\longrightarrow$ elaboration + constraints $\longrightarrow$ explicit core $\longrightarrow$ kernel check.

11.1 Metavariables as questions

Suppose the surface language contains an omitted argument. The elaborator may introduce a metavariable α together with constraints generated by later uses. For example, applying a polymorphic-looking identity combinator to a natural can generate a constraint that the missing domain be $\mathbb{N}$.

A metavariable is not a proof that some term exists. It is a placeholder whose eventual solution must pass the ordinary core checker. This distinction keeps search outside the trusted validity boundary.

11.2 Constraint flow

The small elaborator used here supports only first-order syntactic constraints over explicit type expressions. It performs:

1. decomposition of identical constructors;

2. assignment of an unsolved metavariable to a term that does not contain it;
3. an occurs check to reject cyclic assignments;
4. propagation until no new constraint is produced.

It does not claim complete higher-order unification. Dependent elaboration in production systems is substantially more sophisticated.

11.3 Elaboration may fail even when a term exists

There are at least three distinct failure modes:

Core invalidity. No explicit term satisfying the generated judgment exists in the selected calculus.

Search incompleteness. A valid explicit term exists, but the elaborator’s strategy cannot reconstruct it.

Ambiguity. Multiple solutions satisfy the local constraints and the surface program does not determine which one was intended.

A useful diagnostic should say which kind of failure is known, rather than collapsing all three into “type error.”

Proposition 11.1 (Kernel trust boundary). *If the elaborator outputs an explicit core term t and the kernel checks $\Gamma \vdash t : A$, then the core judgment is valid in the kernel calculus regardless of how the elaborator searched for t .*

The proposition does not prove that t faithfully represents every intended semantics of the original surface text. Correct elaboration is a separate compiler-correctness question.

11.4 Computational Laboratory 11

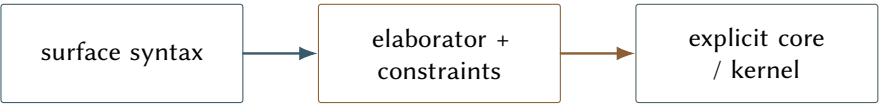
Computational Laboratory

Run `labs/lab11_elaboration.py`. It solves a small set of metavariable constraints, demonstrates occurs-check rejection, records an ambiguous case, and verifies each successful elaboration by passing the reconstructed core type to the checker from Laboratory 10.

11.5 Exercises

11.1. Checkpoint. What object does an elaborator produce for the kernel?

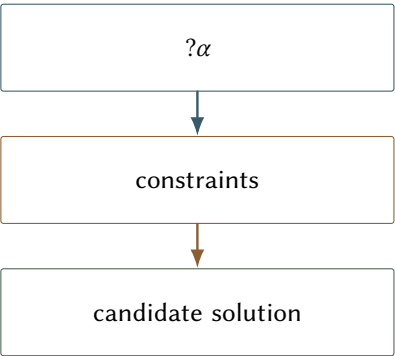
Elaborator and Kernel



Elaboration may be sophisticated while validity remains anchored in a smaller checker.

Figure 11.1: **Plate 31. Elaborator and Kernel.** Elaboration may be sophisticated while validity remains anchored in a smaller checker. Kernel acceptance does not automatically prove faithful surface intent.

Metavariables Carry Constraints

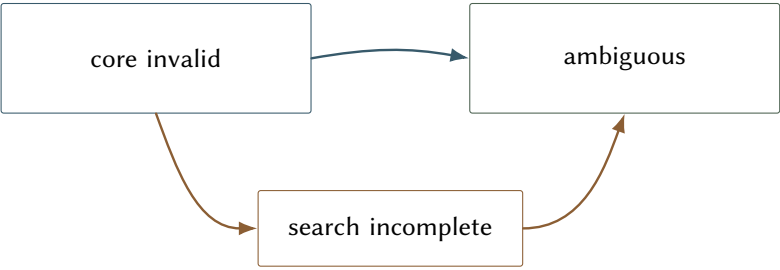


A metavariable is a question constrained by uses, not an accepted proof term.

Figure 11.2: **Plate 32. Metavariables Carry Constraints.** A metavariable is a question constrained by uses, not an accepted proof term. The teaching solver handles only first-order syntactic constraints.

- 11.2. **Checkpoint.** What is a metavariable?
- 11.3. **Checkpoint.** Why is an occurs check needed?
- 11.4. **Core.** Solve the constraints $\alpha = \mathbb{N}$ and $\text{Vec}(\alpha, 2) = \text{Vec}(\mathbb{N}, 2)$.
- 11.5. **Core.** Explain why $\alpha = \Pi(x : \mathbb{N}).\alpha$ must be rejected by an occurs check.
- 11.6. **Core.** Give an ambiguous omitted-argument example.
- 11.7. **Synthesis.** Distinguish elaborator completeness from kernel soundness.
- 11.8. **Synthesis.** Explain why a sophisticated untrusted search procedure can still be compatible with a small trusted checker.
- 11.9. **Proof Workshop.** Prove the kernel trust-boundary proposition from the definition of acceptance.
- 11.10. **Proof Workshop.** Show that first-order constructor decomposition preserves the solution set of an equality constraint.
- 11.11. **Design Clinic.** Design an error report that distinguishes ambiguity, search incompleteness, and core invalidity.
- 11.12. **Challenge.** Explain why dependent implicit-argument inference naturally approaches higher-order unification.

Three Elaboration Failures



Elaboration failure has distinct causes that diagnostics should not collapse.

Figure 11.3: **Plate 33. Three Elaboration Failures.** Elaboration failure has distinct causes that diagnostics should not collapse. A production elaborator may expose additional failure classes.

Chapter 12

Codes for Types, Without a Type of All Types

Once a language can construct many types by formation rules, a natural temptation appears: can the language manipulate descriptions of its own types? This is the threshold of universes. Volume VI will study universes in depth. Here we introduce only enough structure to show why self-description is powerful and why unrestricted self-membership is dangerous.

A Tarski-style preview uses a type of *codes*, say $\mathcal{U}_0$, together with a decoding operation El from codes to types. Example codes might satisfy

$$\text{natC} : \mathcal{U}_0, \quad \text{boolC} : \mathcal{U}_0,$$

with meta-level decoding equations

$$\text{El}(\text{natC}) \equiv \mathbb{N}, \quad \text{El}(\text{boolC}) \equiv \text{Bool}.$$

12.1 Closing codes under type formers

One may add code constructors corresponding to dependent products and sums:

$$\text{piC}(a, b), \quad \text{sigmaC}(a, b),$$

where a codes a domain and b codes a family of result types. Decoding then mirrors the formation rules:

$$\text{El}(\text{piC}(a, b)) \equiv \Pi(x : \text{El}(a)).\text{El}(b(x)).$$

The exact internal syntax of b requires a richer universe presentation than we need here. The lesson is structural: a universe turns formation principles into data that the language can manipulate.

12.2 Why not $\text{Type} : \text{Type}$?

Naively putting the totality of types inside itself recreates paradox pressure. Different type theories avoid this in different ways: cumulative hierarchies, predicativity restrictions, impredicative strata with care, or other semantic/operational designs. Volume II does not prove a paradox theorem for every presentation; it records the design constraint that unrestricted self-classification is not automatically safe.

Do not confuse

A universe code is not “the set of all types.” It is a controlled syntax/semantic interface whose closure and stratification rules must be specified. Writing one symbol Type without those rules hides the central problem.

12.3 Stratification pressure

A minimal hierarchy distinguishes

$$\mathcal{U}_0, \mathcal{U}_1, \mathcal{U}_2, \dots$$

with codes/types at one level classified by a higher one under explicitly stated rules. The hierarchy is not decorative bookkeeping. It is one mechanism for preserving consistency while still allowing generic programming and internal descriptions.

12.4 Computational Laboratory 12

Computational Laboratory

Run `labs/lab12_codes.py`. It implements a tiny external code datatype for $\mathbb{N}$, Bool , products, and vectors, decodes codes to printable core types, and rejects an attempted self-code whose level would need to contain itself. This is a stratification demonstration, not a proof of consistency.

12.5 Exercises

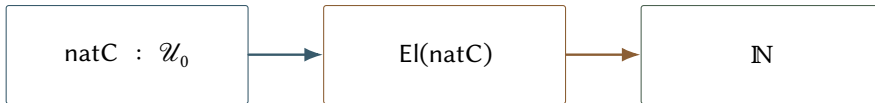
12.1. Checkpoint. What are the two ingredients of a Tarski-style universe preview?

12.2. Checkpoint. State the decoding equations for natural and Boolean codes.

12.3. Checkpoint. Why is “ $\text{Type} : \text{Type}$ ” not adopted silently?

12.4. Core. Design a code for ordinary products and state its decoding equation.

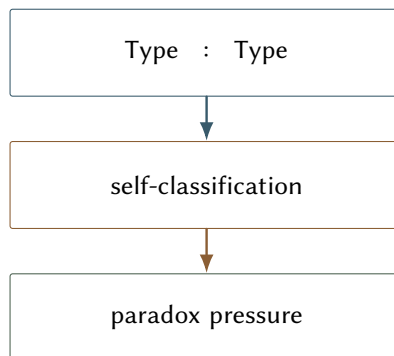
Codes and Decoding



Universe codes make descriptions of types
manipulable under controlled decoding.

Figure 12.1: **Plate 34. Codes and Decoding.** Universe codes make descriptions of types manipulable under controlled decoding. The code language is not the set of all types.

Why Type-in-Type Is Not Free

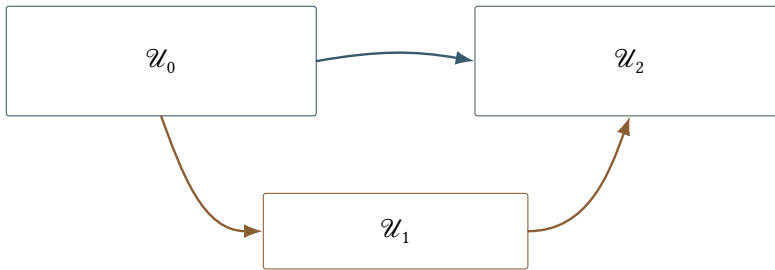


Unrestricted self-classification requires justification and is not silently admitted.

Figure 12.2: **Plate 35. Why Type-in-Type Is Not Free.** Unrestricted self-classification requires justification and is not silently admitted. The plate is a design warning, not a full paradox proof.

- 12.5. **Core.** Design a code for vectors when both the element type and length are explicit external parameters.
- 12.6. **Core.** Explain the difference between a code and the type it decodes to.
- 12.7. **Synthesis.** Explain how universes turn formation principles into manipulable data.
- 12.8. **Synthesis.** Compare Russell-style and Tarski-style presentations at the level of intuition, without claiming them definitionally identical.
- 12.9. **Proof Workshop.** Show by structural induction that the toy decoder is total on the admitted finite code grammar.
- 12.10. **Proof Workshop.** Explain why rejecting self-level classification is a design condition rather than a full consistency proof.
- 12.11. **Design Clinic.** Specify a two-level code hierarchy sufficient to code base types at level 0 and the level-0 code language at level 1.
- 12.12. **Challenge.** Describe what additional metatheory would be needed before claiming a full internal universe hierarchy.

Stratification Pressure



Universe levels control how descriptions of types may themselves be classified.

Figure 12.3: **Plate 36. Stratification Pressure.** Universe levels control how descriptions of types may themselves be classified. Different type theories use different stratification policies.

Chapter 13

Comprehension as Language Design

We can now return to the title of the volume. A computational world is not built merely by listing syntax. It is built by choosing which judgments may be formed, which constructors inhabit them, which eliminators observe them, which computations count definitionally, and which checks remain decidable or tractable. Formation rules determine the space in which later programming is possible.

Two calculi can share the same term notation while admitting different worlds. A calculus with only $A \rightarrow B$ can express fixed function interfaces. Add Π and the result type may depend on the argument. Add Σ and data can package a witness with a witness-dependent payload. Add indexed constructors and some invariants move from external propositions into the shape of admissible terms.

13.1 A categorical reading: context and display map

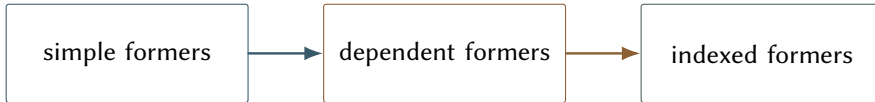
Cartmell’s contextual categories and later comprehension/category-with-families presentations organize dependent syntax categorically. At the level needed here, read a context Γ as an object and a context extension $\Gamma.A$ as equipped with a projection

$$\pi_A : \Gamma.A \rightarrow \Gamma.$$

A substitution/context morphism $\sigma : \Delta \rightarrow \Gamma$ reindexes A to a type $A[\sigma]$ over Δ . The syntactic substitution laws become functorial laws in the semantics.

This viewpoint is powerful because it identifies what is structural across many concrete syntaxes: contexts, reindexing, comprehension/display maps, and operations corresponding to Π and Σ .

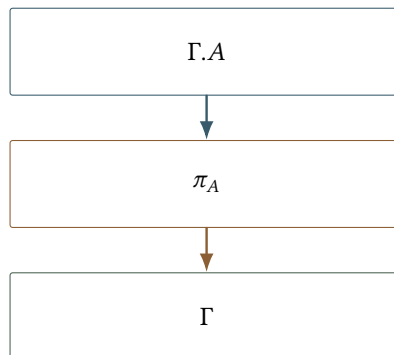
Formation Rules Build Worlds



Changing formation rules changes which computational specifications can be stated directly.

Figure 13.1: **Plate 37. Formation Rules Build Worlds.** Changing formation rules changes which computational specifications can be stated directly. Expressiveness does not automatically imply better engineering.

Comprehension as a Display Map



Categorical comprehension organizes context extension and reindexing.

Figure 13.2: **Plate 38. Comprehension as a Display Map.** Categorical comprehension organizes context extension and reindexing. A model validates syntax; syntax is not literally identical to one model.

Do not confuse

Syntax is not literally a picture of fibers, and a type theory is not automatically identical to one categorical model. A categorical model validates/organizes rules. Different models can support the same syntax, and semantic completeness questions require separate theorems.

13.2 Two calculi, two worlds

Consider two hypothetical languages.

World S has natural numbers, products, sums, and ordinary arrows. It can write a function that takes a vector representation and a runtime length, but the type need not force the representation to agree with the length.

World D adds Π , Σ , and $\text{Vec}(A, n)$. It can state an interface in which the length relation is intrinsic to the type. The gain is precise specification and stronger local checking. The cost is more elaborate substitution, equality, inference, and compiler machinery.

13.3 Expressiveness is not free

This volume therefore weakens any naive version of the series thesis. If type theory is a grammar of computational possibility, then richer grammars do not monotonically improve engineering. They trade representational power against equality complexity, annotation burden, elaboration search, proof effort, and possibly decidability.

A useful unifying claim must be conditional: formation rules expose and constrain computational possibilities, but no single maximal formation language is automatically optimal for every task.

13.4 Computational Laboratory 13

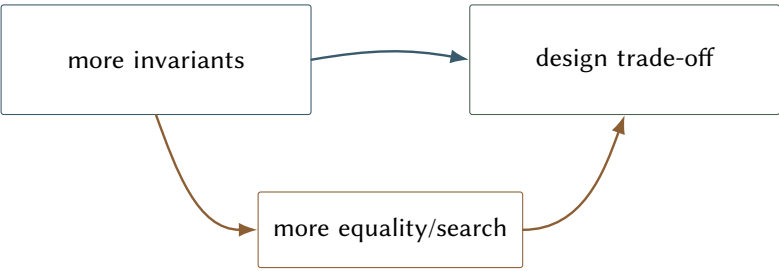
Computational Laboratory

Run `labs/lab13_worlds.py`. It compares two small rule sets over the same examples and records which specifications each can express directly. The lab also attaches a simple “checking burden” count to each feature. The numeric burden is illustrative only; the expressibility differences come from explicit rule availability.

13.5 Exercises

- 13.1. **Checkpoint.** What does a context projection forget?
- 13.2. **Checkpoint.** What operation corresponds syntactically to semantic reindexing?
- 13.3. **Checkpoint.** Name one reason richer dependency can make checking harder.
- 13.4. **Core.** Give an interface expressible directly with Π that cannot be stated as one fixed arrow without externalizing an invariant.
- 13.5. **Core.** Identify the comprehension object associated with $\Gamma \vdash A$ type.
- 13.6. **Core.** State the identity and composition laws expected of substitution/reindexing.
- 13.7. **Synthesis.** Explain what is genuinely shared between telescope syntax and a comprehension-category semantics.
- 13.8. **Synthesis.** Defend or reject the statement “more expressive types are always better.”
- 13.9. **Proof Workshop.** Show how substitution identity/composition motivate functorial reindexing equations.
- 13.10. **Proof Workshop.** Explain why one categorical model does not by itself establish semantic completeness of the syntax.
- 13.11. **Design Clinic.** Compare two type-language designs for tensor shapes: one with runtime checks, one with dependent indices. List benefits and costs.
- 13.12. **Challenge.** State the strongest version of “formation rules build computational worlds” that the evidence of this volume supports without overclaiming.

Expressiveness Has a Cost



Richer dependency trades specification power
against checking and elaboration burden.

Figure 13.3: **Plate 39. Expressiveness Has a Cost.** Richer dependency trades specification power against checking and elaboration burden. The balance is architecture-dependent, not a universal numeric law.

Chapter 14

From Construction to Proposition

The volume began with a modest permission: a type may depend on an earlier term. We can now trace what that permission bought.

A telescope records ordered dependency. Substitution specializes not only terms but the types of later declarations. Π packages a family of result types into one dependent function. Σ packages an index with a payload in the corresponding family member. Indexed constructors make selected invariants part of the terms that can exist. Dependent elimination consumes those invariants without forgetting them. Definitional equality determines which index computations count automatically. Bidirectional checking and elaboration manage the information cost. Universe codes reveal that even formation rules can become data, provided self-description is controlled.

14.1 What the grand-unification thesis has earned

Volume II supports a specific, limited thesis:

Governing Principle

Type formation rules can be read as a grammar of admissible computational worlds: they determine which families may be spoken of, which constructions may inhabit them, and which dependencies a checker can enforce compositionally.

This is stronger than “types prevent errors” and weaker than “all computation is type theory.” The volume has not accounted for effects, open interaction, physical cost, probability, concurrency, undecidability, or semantics independent of syntax. Later volumes will pressure those omissions.

14.2 Indices do not guarantee the world

A type such as $\text{Fin}(n)$ can make an out-of-bound finite index unconstructible in its own calculus. It cannot guarantee that a foreign array really has length n , that memory is not corrupted by an unsafe subsystem, or that the indexed element satisfies a domain-specific semantic invariant. Typed guarantees are relative to the trusted boundary and the propositions encoded.

14.3 The proof/program threshold

Now consider the type

$$\Sigma(n : \mathbb{N}).\text{Vec}(A, n).$$

An inhabitant is ordinary data. But if the family is interpreted as a specification, the same inhabitant is also evidence that some witness n exists together with data satisfying the indexed condition. Likewise, a term of

$$\Pi(x : A).B(x)$$

constructs an output in $B(x)$ for every $x : A$, which begins to resemble a proof of a universally quantified statement.

The resemblance is not yet a theorem because we have not fixed a logic, its proof rules, or the exact correspondence. Volume III must do that work at the rule level.

14.4 A final distinction: specification versus proposition

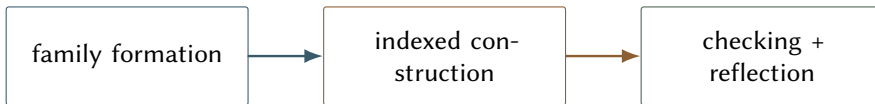
Not every type need be read as a proposition. A vector type can classify data without anyone adopting a logical interpretation. Propositions-as-types is a structural correspondence that becomes precise when logical introduction/elimination and computational construction/elimination are aligned. Treating every type as “a proposition” prematurely obscures the multiple roles types play as data spaces, protocols, resources, effects, and semantic objects.

14.5 Computational Laboratory 14

Computational Laboratory

Run `labs/lab14_threshold.py`. It maps a small set of dependent type constructors to their future logical readings and checks the rule-shape correspondences that will seed Volume III. The output labels these as *correspondence previews*, not logical theorems.

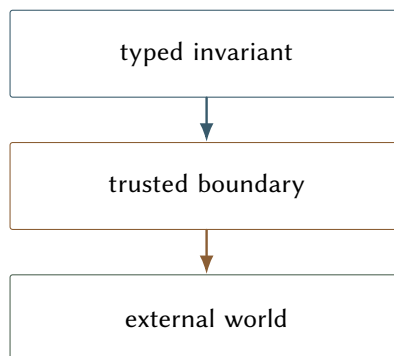
Volume II Synthesis



Dependency propagates from formation through construction, elimination, equality, and checking.

Figure 14.1: **Plate 40. Volume II Synthesis.** Dependency propagates from formation through construction, elimination, equality, and checking. The map is a conceptual dependency graph, not a proof implication graph.

What Indices Cannot Guarantee



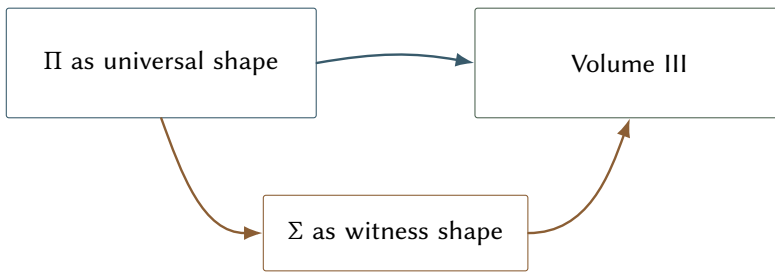
Typed indices guarantee only what is encoded and preserved inside their trust assumptions.

Figure 14.2: **Plate 41. What Indices Cannot Guarantee.** Typed indices guarantee only what is encoded and preserved inside their trust assumptions. Environmental and semantic correctness can require additional evidence.

14.6 Exercises

- 14.1. **Checkpoint.** Name four consequences of allowing type families.
- 14.2. **Checkpoint.** State one guarantee an index can encode and one it cannot.
- 14.3. **Checkpoint.** What is the governing threshold into Volume III?
- 14.4. **Core.** Explain the future existential reading of $\Sigma(n : \mathbb{N}).\text{Vec}(A, n)$ without treating it as already formalized logic.
- 14.5. **Core.** Explain the future universal reading of $\Pi(x : A).B(x)$.
- 14.6. **Core.** List the trusted assumptions behind a claim that a finite index prevents out-of-bounds access.
- 14.7. **Synthesis.** State the strongest unification thesis supported by Volume II and one stronger claim it does not support.
- 14.8. **Synthesis.** Explain why “every type is a proposition” is too crude for the series.
- 14.9. **Proof Workshop.** Match the Π introduction/elimination rules to the natural-deduction shapes expected for universal/implicational reasoning.
- 14.10. **Proof Workshop.** Match the Σ introduction/projection structure to a witness-and-evidence reading, naming the missing logical details.
- 14.11. **Design Clinic.** Choose one invariant from a real software system and describe how far an indexed type could enforce it before environmental assumptions enter.
- 14.12. **Challenge.** Give a concrete counterexample to the naive thesis “if a property can be expressed as a dependent type, the resulting program is correct in every operational sense.”

Threshold to Proof / Program



Dependent construction creates the structural pressure for propositions-as-types.

Figure 14.3: **Plate 42. Threshold to Proof / Program.** Dependent construction creates the structural pressure for propositions-as-types. The logical correspondence is previewed here, not yet established as a theorem.

Formal reference

Judgment forms

$$\Gamma \text{ ctx} \quad \Gamma \vdash A \text{ type} \quad \Gamma \vdash t : A \quad \Gamma \vdash t \equiv u : A.$$

The volume develops four nested teaching fragments: COMP-0 (families and telescopes), COMP- $\Pi\Sigma$ (dependent functions and pairs), COMP-I (selected indexed inductive families), and COMP-CHECK (an explicit bidirectional checker and bounded elaborator). The fragments are pedagogical calculi, not claims to reproduce any one proof assistant kernel.

Core dependent formers

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x : A \vdash B \text{ type}}{\Gamma \vdash \Pi(x : A).B \text{ type}} \quad \frac{\Gamma \vdash A \text{ type} \quad \Gamma, x : A \vdash B \text{ type}}{\Gamma \vdash \Sigma(x : A).B \text{ type}},$$

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x. b : \Pi(x : A).B} \quad \frac{\Gamma \vdash f : \Pi(x : A).B \quad \Gamma \vdash a : A}{\Gamma \vdash fa : B[a/x]},$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B[a/x]}{\Gamma \vdash \langle a, b \rangle : \Sigma(x : A).B}.$$

Notation guide

Notation	Meaning
Γ, Δ	dependent contexts/telescopes
$B(x)$	a type family that may mention x
$\Pi(x : A).B$	dependent function type
$\Sigma(x : A).B$	dependent pair type
$t[a/x]$	capture-avoiding substitution
$\text{Fin}(n)$	finite index family
$\text{Vec}(A, n)$	vectors of elements of A indexed by length
$\equiv$	definitional equality in the current fragment

$\Gamma \vdash t \Rightarrow A$	algorithmic synthesis
$\Gamma \vdash t \Leftarrow A$	algorithmic checking
$\mathcal{U}_0, \text{El}$	preview universe of codes and decoding operation

Glossary

Comprehension

Context extension by a well-formed type family; later given a categorical reading through display/context projections.

Dependent family

A type expression well formed under a context containing an index variable.

Telescope

An ordered dependent context in which later declarations may depend on earlier declarations.

Motive

The family describing the result type of a dependent eliminator.

Indexed family

A family whose index records a static invariant, such as vector length.

Conversion

Typing up to the fragment's definitional equality.

Elaboration

Reconstruction of omitted information by producing a more explicit core term plus constraints.

Universe code

A term that denotes/describes a type under a decoding operation; not automatically a “type of all types.”

Bibliography and historical notes

The historical remarks in this volume are deliberately modest. Martin-Löf's 1984 lecture notes are used as a primary reference for the judgmental presentation of dependent type theory; de Bruijn's AUTOMATH work supplies earlier evidence that dependent formal languages were practical objects of study; Cartmell's contextual categories, Dybjer's work on inductive families, and Hofmann and Jacobs on dependent-type semantics anchor later developments. The bibliography audit records the metadata checked for the release candidate.

Bibliography

- [1] Per Martin-Löf. *Intuitionistic Type Theory*. Notes by Giovanni Sambin. Bibliopolis, Naples, 1984.
- [2] N. G. de Bruijn. “The mathematical language AUTOMATH, its usage and some of its extensions.” In *Symposium on Automatic Demonstration*, Lecture Notes in Mathematics 125, pp. 29–61. Springer, 1970. DOI: 10.1007/BFb0060623.
- [3] John Cartmell. *Generalised Algebraic Theories and Contextual Categories*. D.Phil. thesis, University of Oxford, 1978.
- [4] Thierry Coquand and Gérard Huet. “The Calculus of Constructions.” *Information and Computation* 76(2–3):95–120, 1988. DOI: 10.1016/0890-5401(88)90005-3.
- [5] Peter Dybjer. “Inductive families.” *Formal Aspects of Computing* 6(4):440–465, 1994. DOI: 10.1007/BF01211308.
- [6] Bengt Nordström, Kent Petersson, and Jan M. Smith. *Programming in Martin-Löf’s Type Theory: An Introduction*. Oxford University Press, 1990.
- [7] Martin Hofmann. “Syntax and Semantics of Dependent Types.” In A. M. Pitts and P. Dybjer, eds., *Semantics and Logics of Computation*, pp. 79–130. Cambridge University Press, 1997. DOI: 10.1017/CBO9780511526619.004.
- [8] Bart Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141. Elsevier, 1999.
- [9] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [10] Robert Harper. *Practical Foundations for Programming Languages*. 2nd ed. Cambridge University Press, 2016.
- [11] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. Used only for later perspective; Volume II does not develop identity types or univalence.

Course and self-study roadmap

A fourteen-week course can follow one chapter per week. A ten-week course can combine Chapters 1–2, 6–7, 10–11, and 12–13. The computational laboratories are intentionally small enough to run on an ordinary laptop. For self-study, the recommended rhythm is: read the conceptual opening, reproduce the central derivation by hand, run the chapter laboratory, then attempt the two Proof Workshop exercises before consulting the solutions companion.

Transition to Volume III

A dependent type is still, formally, a type. Yet by the end of this volume its inhabitants routinely function as evidence that an indexed condition can be met: a value of $\text{Fin}(n)$ certifies a usable position, a value of $\text{Vec}(A, n)$ certifies a length invariant, and a dependent pair packages a witness together with information whose type depends on that witness.

This creates the next unavoidable question. If constructing a term can amount to satisfying a specification, when does a type become a proposition and an inhabitant become a proof? The answer cannot be a slogan. It requires a rule-level comparison between logical introduction/elimination and computational construction/reduction. That is the work of Volume III, *PROOF / PROGRAM — Logic Becomes Executable*.

Index

- algorithmic equality, 47
- algorithmic soundness, 53
- annotation, 53
- beta reduction, 14
- bidirectional typing, 53
 - Pi, 14
- canonical forms
 - Fin, 32
 - vector, 37
- capture avoidance, 8
- categorical semantics, 68
- checker
 - dependent, 53
- code and decoding, 63
- comprehension, 26
- comprehension category, 68
- congruence, 47
- constraint solving, 58
- constructor refinement, 32
 - vector, 37
- context
 - dependent, 2
- context extension, 26
- context morphism, 26
- context projection, 26
- contextual category, 68
- conversion rule, 47
- definitional equality, 47
- dependent elimination, 42
- dependent function, 14
- dependent pair, 20
- dependent product, 14
- dependent sum, 20
- dependent type, 2
- display map, 68
- elaboration, 58
- elaborator, 58
- eliminator, 42
- exchange
 - conditional, 26
- existential reading, 20
- expressiveness trade-off, 68
- Fin type, 32
- finite type, 32
- formation rule, 2
- function type
 - constant family, 14
- generalized substitution, 8
- higher-order unification, 58
- indexed family, 32
- inductive family, 37
- kernel trust boundary, 58
- language design, 68
- length index, 37
- metavariable, 58
- motive, 42
- occurs check, 58

- Pi type, 14
- product type
 - constant family, 20
- projection
 - dependent, 20
- proof/program correspondence
 - preview, 73
- propositional equality
 - distinction, 47
- propositions-as-types
 - threshold, 73
- reindexing, 8
- safe lookup, 42
- series thesis, 73
- Sigma type, 20
- specification, 73
- static index, 32
- stratification, 63
- strict positivity, 37
- structural rule, 26
- substitution
 - composition, 8
 - dependent, 8
- Tarski-style universe, 63
- telescope, 2, 3
- totality
 - local guarantee, 42
- trailing context, 8
- transport pressure, 47
- trusted boundary, 73
- type checking, 53
- type family, 2
- type synthesis, 53
- Type-in-Type, 63
- universe
 - preview, 63
- universe hierarchy, 63
- vector head, 42
- vector type, 37
- Volume III transition, 73
- weakening, 2